



UNIVERSITÀ
DEGLI STUDI
FIRENZE

UNIVERSITÀ DEGLI STUDI DI FIRENZE

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

SafeCore

Password Manager Sicuro

Autori:

Dania CIAMPALINI

Cecilia VIGNANI

Docente:

Prof. Enrico VICARIO

Corso:

Ingegneria del Software



[GitHub Repository](#)

Gennaio-Marzo 2026

Indice

1	Motivazione e Descrizione	9
1.0.1	Funzionalità principali	9
1.0.2	Stack tecnologico	9
1.0.3	Possibili aggiunte e sviluppi futuri	10
2	Analisi dei Requisiti e Casi d'Uso	11
2.1	Use Case Diagram	11
2.1.1	Attori	12
2.1.2	Use Case Template	12
2.1.3	Mockups	17
2.1.4	Requisiti non funzionali	22
3	Progettazione e Implementazione	27
3.1	Scelte progettuali e considerazioni architetturali	27
3.2	Architettura del Sistema (Component Diagram)	27
3.2.1	Analisi dei Macro-Componenti	28
3.2.2	Analisi del Component Diagram	28
3.3	Gestione dei Dati (Database)	29
3.3.1	Modellazione dei Dati (Schema ER)	29
3.3.2	Traduzione nel Modello Relazionale (PostgreSQL)	30
3.4	Design e Implementazione (Class Diagram e Interfacce)	31
3.5	Organizzazione dei Package e Compiti dei Layer	32
3.5.1	Analisi delle Entità (Domain Model)	33
3.6	Analisi Dettagliata dei Componenti per Package	33
3.6.1	Package <code>com.safecore.ui</code> (Fig. 3.3)	34
3.6.2	Package <code>com.safecore.business</code> (Fig. 3.4)	34
3.6.3	Package <code>com.safecore.security</code> (Fig. 3.5)	34
3.6.4	Package <code>com.safecore.persistence</code> (Fig. 3.6)	34
3.6.5	Sintesi delle Responsabilità dei Componenti	35
3.7	Dinamica del sistema	36
3.7.1	Analisi dei flussi principali	36
3.7.2	Principi di Design	37
3.8	Idiomi e Design Pattern	38
3.8.1	Copia Difensiva (Defensive Copy)	38
3.8.2	Strategy Pattern (Cifratura)	38
3.8.3	Observer Pattern (Sincronizzazione Reattiva)	39
3.8.4	Builder e Repository	40
3.8.5	Singleton e Factory	42

4	Testing	45
4.1	Strategia di Verifica	45
4.1.1	Tecnologie e Strumenti	46
4.1.2	Configurazione e Annotazioni	46
4.2	Analisi dei Livelli di Testing	46
4.2.1	Livello 1: Domain Layer e Logica Pura (Unit Test)	46
4.2.2	Livello 2: Business Logic e Service Layer (Mock Testing)	47
4.2.3	Livello 3: Persistence Layer e Testing dei DAO	49
4.2.4	Livello 4: Test Funzionali e di Integrazione	49
4.2.5	Livello 5: Interfaccia Utente e TestFX	51
4.3	Analisi della Copertura e Risultati Finali	51
4.3.1	Statistiche della Suite di Test	51
5	Qualità del Software	53
5.1	Rispetto dei Principi SOLID	53
5.2	Analisi delle Performance	53
5.3	Requisiti Funzionali e Non Funzionali	53
5.4	Metriche di mantenibilità	54

Elenco delle figure

2.1	Diagramma degli Use Case	11
2.2	Caratteristiche della qualità del prodotto software (ISO/IEC 25010).	23
3.1	Diagramma dei Componenti	28
3.2	Schema ER per la modellazione dei dati nel database.	29
3.3	Class Diagram - Parte 1: Livello di Presentazione (UI, Navigation, Session).	31
3.4	Class Diagram - Parte 2: Livello di Business (Services, Domain, Hints, Exceptions).	31
3.5	Class Diagram - Parte 4: Modulo di Sicurezza (Strategies, Key Management).	32
3.6	Class Diagram - Parte 3: Layer di Persistenza (Entities, Repositories).	32
3.7	Package Diagram	33
3.8	Diagramma di sequenza del processo di Login.	36
3.9	Diagramma di sequenza del salvataggio di una Password.	37
3.10	Pattern Strategy applicato alla cifratura dei dati.	39
3.11	Pattern Observer: disaccoppiamento tramite inversione delle dipendenze.	39
4.1	Piramide del Testing: alla base ci sono i test più numerosi, veloci ed economici. La sicurezza e le prestazioni vengono verificate come dimensioni trasversali all'intera struttura.	45

Elenco dei Mockups

1	Mockup della Registrazione Nuovo Utente, UC-1.	17
2	Mockup del Login dell'Utente Registrato, UC-2.	17
3	Mockup della schermata principale del Vault.	18
4	Mockup dell'Inserimento Nuova Credenziale, UC-3.	18
5	Mockup della Generazione Password Sicura, UC-4.	19
6	Mockup dell'Esportazione Backup Vault, UC-5.	19
7	Mockup dell'Invio di Segreto SafeSend.	20
8	Mockup dell'Accesso a Segreto SafeSend, UC-6.	20
9	Mockup dell'Audit Sicurezza Vault, UC-7.	21
10	Mockup della Richiesta di eliminazione account.	21
11	Mockup dell'Eliminazione Account, UC-8	22

Capitolo 1

Motivazione e Descrizione

SafeCore è un gestore di password locale e sicuro. Si tratta di un Software applicativo desktop in Java (più propriamente un pezzo di software) che rientra nelle categoria delle utility dedicate alla sicurezza informatica. SafeCore lavora interamente offline, utilizza algoritmi di crittografia standard industriale (AES e BCrypt) con lo scopo di proteggere dati e password personali con un alto livello di sicurezza.

1.0.1 Funzionalità principali

Le tre funzionalità principali integrate sono le seguenti:

- **Cifratura del Vault:** gestione di un archivio protetto mediante algoritmo **AES-256** in modalità CBC. Ogni password viene cifrata singolarmente prima della memorizzazione, garantendo che l'informazione resti inaccessibile anche in caso di compromissione del sistema di memorizzazione.
- **Security Audit:** un sistema di analisi che valuta la sicurezza delle password e la loro vulnerabilità basandosi su criteri di complessità, lunghezza e data di scadenza, fornendo suggerimenti per rafforzare il profilo di sicurezza.
- **SafeSend:** un sistema di condivisione di segreti basato su link univoci con logica di autodistruzione e *Time-To-Live* (TTL) configurabile, utile per lo scambio sicuro di informazioni sensibili tra soggetti diversi.

1.0.2 Stack tecnologico

Le scelte implementative sono state guidate dalla necessità di garantire portabilità, mantenibilità e robustezza, in modo da progettare un sistema in grado di evolvere nel tempo senza compromettere la coerenza architetturale.

Elementi tecnici di sviluppo

In quest'ottica, il linguaggio di sviluppo **Java 17** all'interno dell'IDE **IntelliJ** JetBrains è stato integrato dal sistema di *Dependency Injection* di **Spring Boot** per la gestione strutturata ed automatizzata del ciclo di vita di componenti e servizi a favore di un basso accoppiamento tra i moduli applicativi.

La scelta per l'interfaccia grafica è ricaduta sulla libreria grafica **Java FX** adottando il pattern architetturale MVC, allo scopo di mantenere una netta separazione tra logica di business e presentazione.

La persistenza dei dati è stata implementata attraverso un database relazionale sviluppato mediante **PostgreSQL** e successivamente migrato su infrastruttura cloud attraverso la piattaforma **Neon**. L'interazione tra l'applicazione e il database è mediata da **Spring Data JPA**, che sfrutta l'ORM **Hibernate** per mappare le classi Java sulle tabelle relazionali, semplificando la gestione di alto livello delle transazioni e riducendo la ridondanza implementativa legata alla gestione manuale dei dati. Per la connessione a basso livello tra il framework e Neon, è stato utilizzato il driver **JDBC** specifico per PostgreSQL, garantendo l'efficienza delle query e la gestione del pool di connessioni.

Infine, la sicurezza crittografica dell'applicativo si basa sulle librerie standard **JCE** (*Java Cryptography Extension*) per l'utilizzo dell'algoritmo AES di cifratura delle password e sull'algoritmo **BCrypt** per la derivazione sicura degli hash delle Master Password e la loro memorizzazione codificata.

Strumenti aggiuntivi

Per lo sviluppo del codice ci si è serviti di strumenti di Intelligenza Artificiale Generativa quali **Gemini** e **Github Copilot** per la generazione di alcune parti di codice e aiuto nel debug, mentre per la modellazione grafica all'interno della presente relazione sono stati utilizzati **Diagrams.net**, **PlantUML** e **TikZ** per la creazione di diagrammi UML. In ultimo, è stato utilizzato **JaCoCo** per l'analisi della copertura del codice, per avere la garanzia che le logiche critiche di cifratura e accesso ai dati fossero opportunamente verificate dai test unitari.

1.0.3 Possibili aggiunte e sviluppi futuri

Il sistema è stato progettato con un'architettura modulare per favorire future aggiunte di funzionalità o evoluzioni tecnologiche.

Integrazione TOTP (2FA)

L'espansione delle funzionalità del vault per includere la generazione di codici *Time-based One-Time Password* o per prevedere sistemi di autenticazione a due fattori (*Two-Factor Authentication*) trasformerebbe SafeCore in un hub di autenticazione completo, eliminando la necessità di applicazioni di terze parti.

Monitoraggio Data Breach

Integrazione con servizi di intelligence esterni per la verifica automatizzata delle credenziali. Il sistema potrebbe segnalare in tempo reale se una password salvata è apparsa in leak pubblici noti, forzando la rotazione della stessa per prevenire attacchi di *credential stuffing*.

Sblocco Biometrico Avanzato

Implementare il supporto nativo a sistemi come FaceID o impronta digitale. Renderebbe l'accesso quotidiano molto più fluido e rapido, pur mantenendo l'opzione sicura della Master Password come ancora di sicurezza per i dati cifrati.

Capitolo 2

Analisi dei Requisiti e Casi d'Uso

In questo capitolo si descrivono i requisiti del sistema attraverso l'uso di diagrammi e template descrittivi, analizzando le interazioni tra gli attori e il software SafeCore.

2.1 Use Case Diagram

Il diagramma dei casi d'uso (Figura 2.1) fornisce una visione d'insieme delle funzionalità offerte e dei processi di sicurezza necessari per l'operatività del vault.

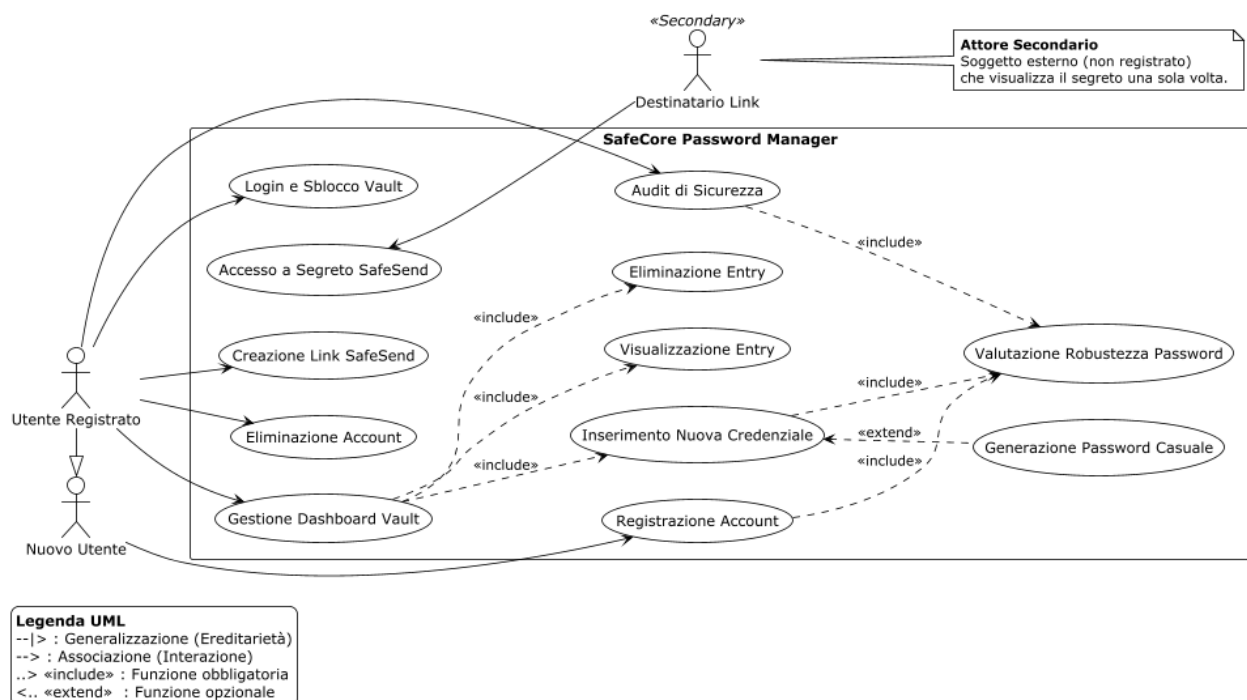


Figura 2.1: Diagramma degli Use Case

2.1.1 Attori

Il sistema *SafeCore* distingue gli attori in base al loro ruolo operativo e alle autorizzazioni d'accesso richieste per le diverse funzionalità.

Attori Primari (Gestione Account e Vault):

- **New User:** l'utente in fase di registrazione iniziale, che interagisce con i processi di creazione del vault e hashing della Master Password.
- **User:** l'utente autenticato che ha pieno accesso alla gestione delle proprie credenziali, all'audit di sicurezza e altre funzionalità.

Attori Secondari (Fruizione Servizi):

- **Destinatario Link (Link Recipient):** È l'unico attore associato alla visualizzazione dei segreti condivisi. È definito come attore secondario poiché la sua interazione è limitata, temporanea e non richiede il possesso di un account.

Nota sulla sovrapposizione dei ruoli: sebbene un **Utente Registrato** possa tecnicamente ricevere e decifrare un link *SafeSend* all'interno della propria applicazione, a livello di modellazione tale azione ricade sotto la definizione di **Destinatario Link**. Questo approccio permette di mantenere una netta separazione tra la gestione persistente del Vault (che richiede la Master Password) e il consumo volatile dei segreti condivisi (che richiede solo il token del link).

2.1.2 Use Case Template

Per garantire una descrizione dettagliata delle funzionalità del sistema, vengono riportate di seguito le tabelle relative ai casi d'uso UC1-UC8 più significativi.

UC 1	Registrazione Nuovo Utente
Livello	User Goal
Descrizione	Permettere a un nuovo utente di creare un profilo protetto da Master Password.
Attori	Nuovo Utente
Pre-condizioni	Applicazione avviata; nessun utente autenticato.
Scenario Principale	1. L'utente inserisce email e Master Password. 2. Il sistema valida la robustezza della password tramite criteri minimi di sicurezza. 3. Il sistema verifica l'unicità dell'email nel database. 4. Il sistema genera un salt univoco (32 byte) per la futura derivazione della chiave. 5. Il sistema esegue l'hashing della password e memorizza il profilo utente completo.
Scenari Alternativi	2.a Password troppo debole: il sistema segnala il mancato raggiungimento dei requisiti e blocca la procedura. 3.a Email già esistente: il sistema nega la registrazione per prevenire account duplicati.
Post-condizioni	Account creato correttamente; l'utente viene reindirizzato alla schermata di login.
Interfaccia (Mockup)	Mck. 1

Tabella 2.1: Use Case 1: Registrazione Nuovo Utente.

UC 2	Login e Sblocco Vault
Livello	User Goal
Descrizione	Autenticare l'utente e inizializzare la chiave di cifratura per la sessione corrente.
Attori	Utente Registrato
Pre-condizioni	Account esistente; database raggiungibile.
Scenario Principale	1. L'utente inserisce email e Master Password. 2. Il sistema recupera i dati dell'utente e verifica la validità della password. 3. Il sistema utilizza la password e il salt memorizzato per derivare la chiave AES-256 (PBKDF2). 4. Il sistema inizializza il contesto di sessione e carica la Dashboard. 5. Il sistema decifra le voci del vault per la visualizzazione immediata.
Scenari Alternativi	2.a Credenziali errate: il sistema nega l'accesso con un messaggio generico (protezione da enumerazione). 5.a Decifratura fallita: il sistema avvisa dell'impossibilità di sbloccare i dati (possibile corruzione del vault).
Post-condizioni	Sessione attiva; chiave di cifratura disponibile in memoria RAM volatile.
Interfaccia (Mockup)	Mck. 2

Tabella 2.2: Use Case 2: Login e Sblocco Vault.

UC 3	Inserimento Nuova Credenziale
Livello	User Goal
Descrizione	Aggiungere un nuovo segreto cifrato all'archivio dell'utente.
Attori	Utente Autenticato
Pre-condizioni	Sessione attiva; Dashboard visualizzata.
Scenario Principale	1. L'utente inserisce i dettagli del servizio (nome, username, password). 2. Il sistema genera un IV (Vettore di Inizializzazione) casuale di 16 byte. 3. Il sistema cifra la password utilizzando la chiave di sessione e l'IV generato. 4. Il record viene persistito nel database associandolo univocamente all'utente.
Scenari Alternativi	1.a Campi obbligatori mancanti: il sistema evidenzia gli input non validi e impedisce il salvataggio. 4.a Errore di persistenza: il sistema informa l'utente e mantiene i dati nel form per un nuovo tentativo.
Post-condizioni	Nuovo segreto archiviato in modo sicuro; Dashboard aggiornata con la nuova voce.
Interfaccia (Mockup)	Mck. 4

Tabella 2.3: Use Case 3: Inserimento Nuova Credenziale.

UC 4	Generazione Password Sicura
Livello	Subfunction
Descrizione	Fornire all'utente una stringa casuale ad alta entropia per nuovi segreti.
Attori	Utente Autenticato
Pre-condizioni	Schermata di creazione credenziale aperta.
Scenario Principale	1. L'utente definisce lunghezza e parametri di complessità (simboli, numeri). 2. Il sistema genera una stringa casuale utilizzando un algoritmo di campionamento non distorto. 3. La password viene mostrata nel campo di testo con l'opzione di copia negli appunti.
Scenari Alternativi	1.a Parametri non sicuri: il sistema suggerisce una configurazione minima per prevenire password deboli. 3.a Protezione Clipboard: il sistema notifica che i dati negli appunti verranno rimossi dopo un breve intervallo.
Post-condizioni	Password generata e disponibile per l'uso immediato nel form di inserimento.
Interfaccia (Mockup)	Mck. 5

Tabella 2.4: Use Case 4: Generazione Password Sicura.

UC 5	Esportazione Backup Vault
Livello	User Goal
Descrizione	Creare un archivio cifrato portabile protetto da una password dedicata.
Attori	Utente Autenticato
Pre-condizioni	Sessione attiva; presenza di dati nel vault.
Scenario Principale	1. L'utente avvia l'esportazione. 2. Il sistema decifra le entry correnti e prepara la struttura dati per l'export. 3. Il sistema deriva una nuova chiave. 4. I dati vengono cifrati con la nuova chiave e salvati in un file con estensione .safe.
Scenari Alternativi	3.a Errore di scrittura: il sistema segnala l'impossibilità di accedere alla destinazione file scelta.
Post-condizioni	File di backup generato; i dati sono ora protetti indipendentemente dalla Master Password.
Interfaccia (Mockup)	Mck. 6

Tabella 2.5: Use Case 5: Esportazione Backup Vault.

UC 6	Accesso a Segreto SafeSend
Livello	User Goal
Descrizione	Permettere a un destinatario esterno di visualizzare un segreto una sola volta.
Attori	Destinatario Link
Pre-condizioni	Possesso di un link di condivisione valido e attivo.
Scenario Principale	1. Il destinatario apre il link ricevuto tramite browser. 2. Il sistema verifica l'esistenza del segreto e la validità del token di accesso. 3. Il sistema controlla che il segreto non sia scaduto temporalmente. 4. Il sistema decifra il messaggio, lo mostra e lo elimina istantaneamente dal database.
Scenari Alternativi	2.a Link non trovato: il sistema informa che il segreto è inesistente o già stato consumato. 3.a Link scaduto: il sistema notifica il superamento del limite temporale e rimuove il record.
Post-condizioni	Segreto visualizzato dal destinatario; record rimosso permanentemente (Burn After Reading).
Interfaccia (Mockup)	Mck. 8

Tabella 2.6: Use Case 6: Accesso a Segreto SafeSend.

Nota sull'implementazione UC6:

Sebbene l'attuale prototipo di SafeCore sia un'applicazione desktop, la funzionalità SafeSend è stata progettata per generare link che puntano a una risorsa web. Nella nostra visione architetturale, il sistema prevede un piccolo modulo web disaccoppiato che serve esclusivamente a decifrare i segreti SafeSend in modalità Zero-Knowledge nel browser del destinatario.

UC 7	Audit Sicurezza Vault
Livello	User Goal
Descrizione	Analizzare lo stato di sicurezza complessivo dell'archivio password.
Attori	Utente Autenticato
Pre-condizioni	Sessione attiva; vault popolato.
Scenario Principale	1. L'utente richiede l'analisi di sicurezza dalla Dashboard. 2. Il sistema decifra le password e le analizza statisticamente (lunghezza, entropia). 3. Il sistema identifica duplicati (tramite confronto di hash) e password obsolete. 4. Il sistema calcola un punteggio di salute (Health Score) basato sulle criticità rilevate. 5. Il sistema presenta un report dettagliato con raccomandazioni per ogni voce critica.
Scenari Alternativi	2.a Errore analisi: il sistema segnala se alcune entry non possono essere valutate correttamente. 4.a Vault ottimale: il sistema conferma il raggiungimento del massimo punteggio di sicurezza.
Post-condizioni	L'utente ottiene una panoramica chiara dei punti deboli del proprio archivio.
Interfaccia (Mockup)	Mck. 9

Tabella 2.7: Use Case 7: Audit Sicurezza Vault.

UC 8	Eliminazione Sicura Account
Livello	User Goal
Descrizione	Rimuovere permanentemente l'identità e i dati dell'utente dal sistema.
Attori	Utente Autenticato
Pre-condizioni	Utente loggato; accesso alla sezione impostazioni.
Scenario Principale	1. L'utente attiva la procedura di cancellazione e conferma la comprensione del rischio. 2. L'utente inserisce la stringa di conferma "Elimina il mio account SafeCore" e la password attuale. 3. Il sistema valida la richiesta e procede alla sanitizzazione dei dati. 4. Le entry del vault e il profilo utente vengono sovrascritti con dati casuali prima della cancellazione. 5. Il sistema invalida la sessione e bonifica la memoria RAM dalle chiavi residue.
Scenari Alternativi	2.a Conferma errata: la procedura viene interrotta se il testo o la password non corrispondono. 4.a Fallimento atomicità: in caso di errore, il sistema esegue il rollback per evitare stati inconsistenti.
Post-condizioni	Account e dati distrutti permanentemente; impossibilità tecnica di recupero post-eliminazione.
Interfaccia (Mockup)	Mck. 11

Tabella 2.8: Use Case 8: Eliminazione Sicura Account.

Si riporta qui sotto la Matrice di Tracciabilità per mostrare il collegamento tra ciascun UC e le sue classi di testing (Cap. 4.2.2).

UC	Titolo	Stato Test	File di Riferimento
UC1	Registrazione	Completo	<code>UserServiceTest.java</code> , <code>UserRepositoryTest.java</code>
UC2	Login e Sblocco	Completo	<code>UserServiceTest.java</code> , <code>SafeCoreIntegTest.java</code>
UC3	Nuova Credenziale	Completo	<code>VaultServiceTest.java</code> , <code>PasswordServiceTest.java</code>
UC4	Generatore PW	Completo	<code>PasswordGeneratorTest.java</code>
UC5	Backup	Completo	<code>BackupIntegrationTest.java</code>
UC6	SafeSend	Eccellente	<code>SafeSendServiceTest.java</code> , <code>SafeSendRepositoryTest.java</code>
UC7	Security Audit	Completo	<code>SecurityAuditServiceTest.java</code>
UC8	Eliminazione	Completo	<code>UserServiceTest.java</code> (metodo <code>secureDeleteAccount</code>)

Tabella 2.9: Matrice di tracciabilità tra Casi d'Uso e suite di test.

2.1.3 Mockups

In questa fase sono stati realizzati i mockup dell'interfaccia grafica per verificare la *User Experience* e garantire che tutte le funzionalità descritte nei casi d'uso siano accessibili in modo immediato ed intuitivo.

SafeCore - Registrazione

Crea Account

Inizia a proteggere i tuoi segreti oggi

Email

Master Password

 Show

SICURA (Forte)

Conferma Master Password

 Show

Registrati Ora

[Genera Password Suggestita](#)

IMPORTANTE: Architettura Zero-Knowledge
SafeCore non memorizza la tua password d'accesso e non può aiutarti a recuperarla

[Hai già un account? Accedi](#)

Mck. 1: Mockup della Registrazione Nuovo Utente, UC-1.

SafeCore - Login

SafeCore

Proteggi la tua identità digitale

Email

Master Password

 Show

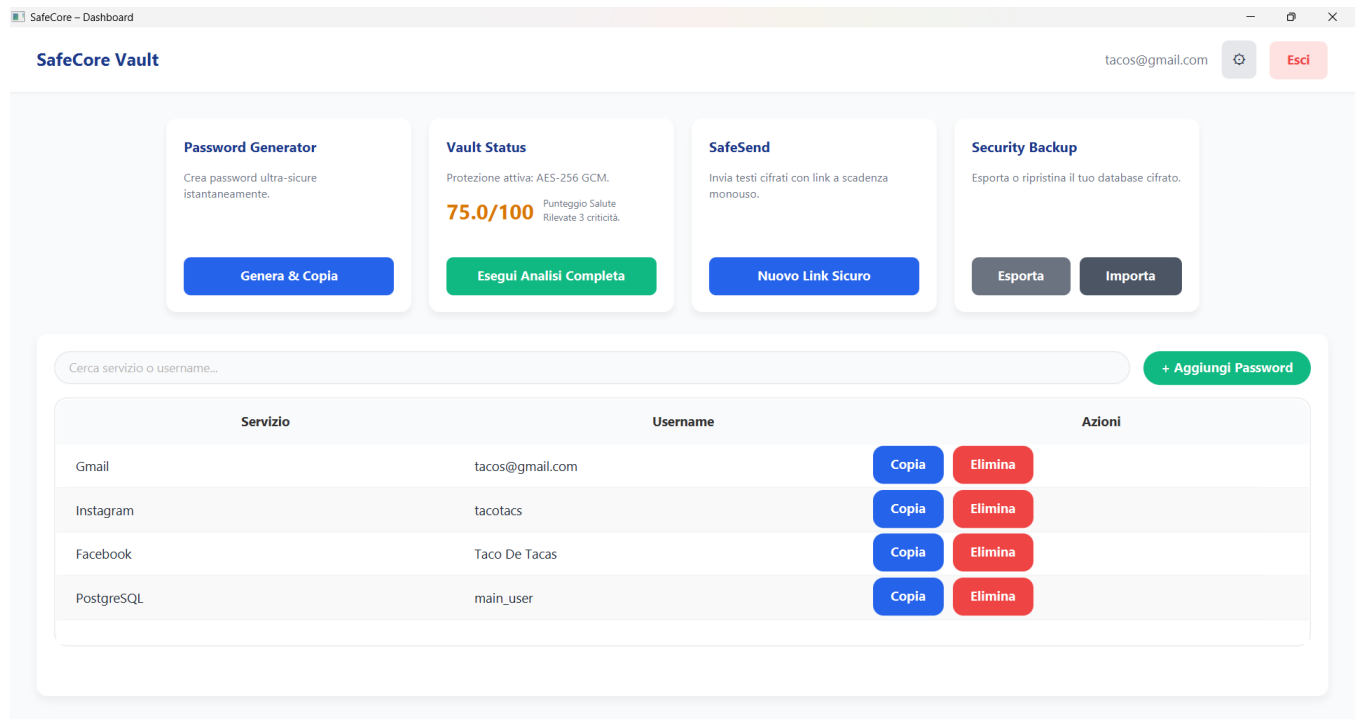
Accedi al Vault

Nuovo utente?

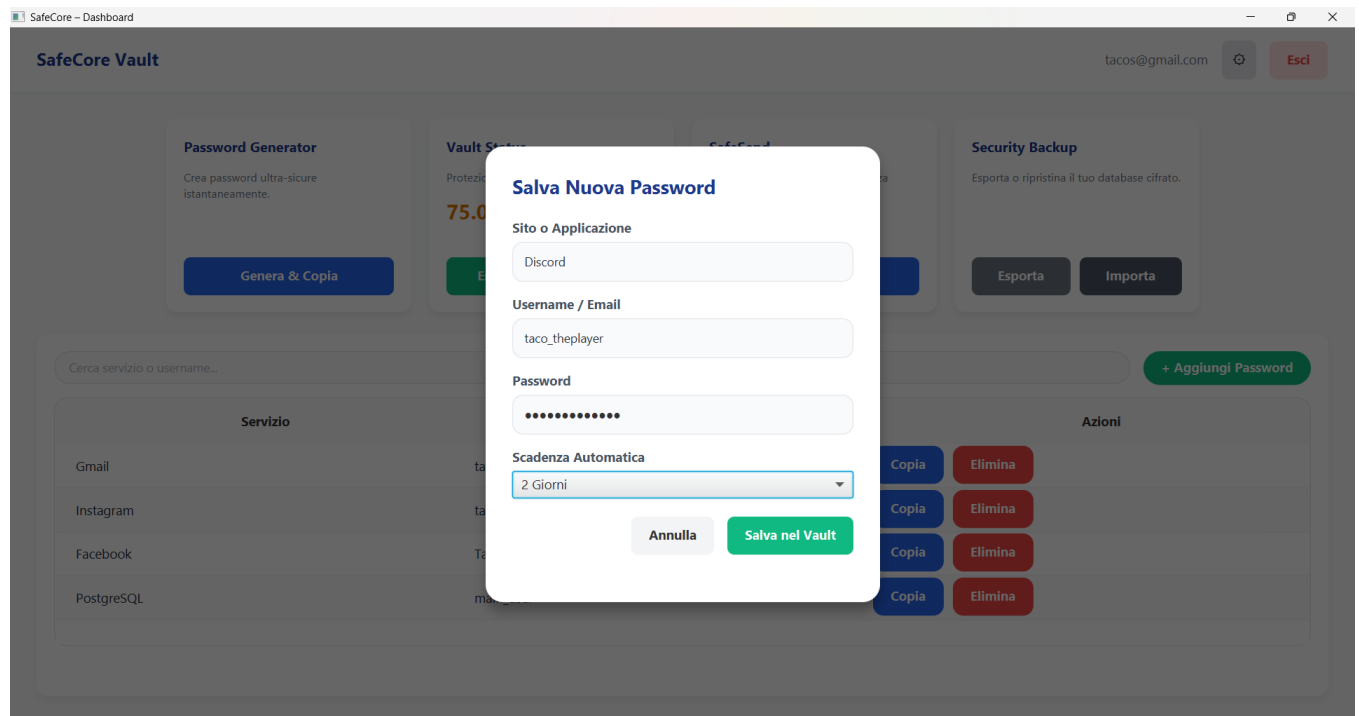
[Crea un nuovo account](#)

IMPORTANTE: Architettura Zero-Knowledge
SafeCore non memorizza la tua password d'accesso e non può aiutarti a recuperarla

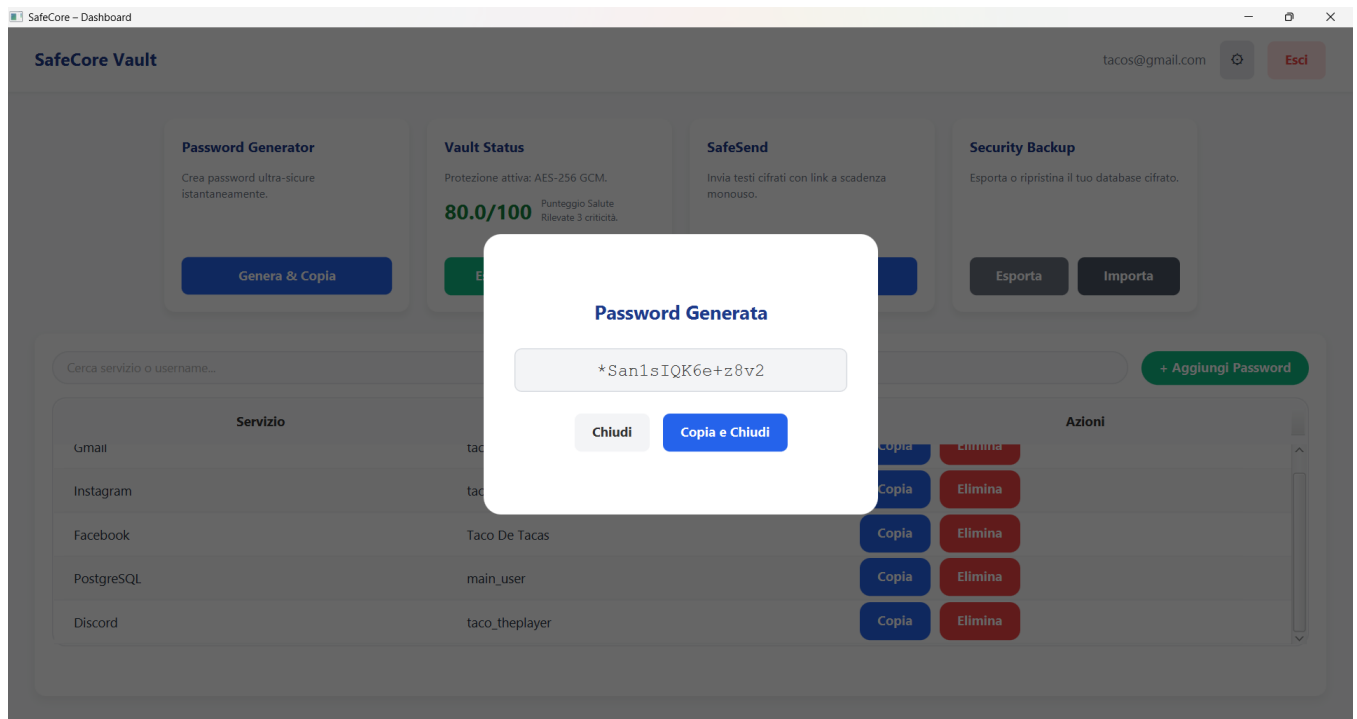
Mck. 2: Mockup del Login dell'Utente Registrato, UC-2.



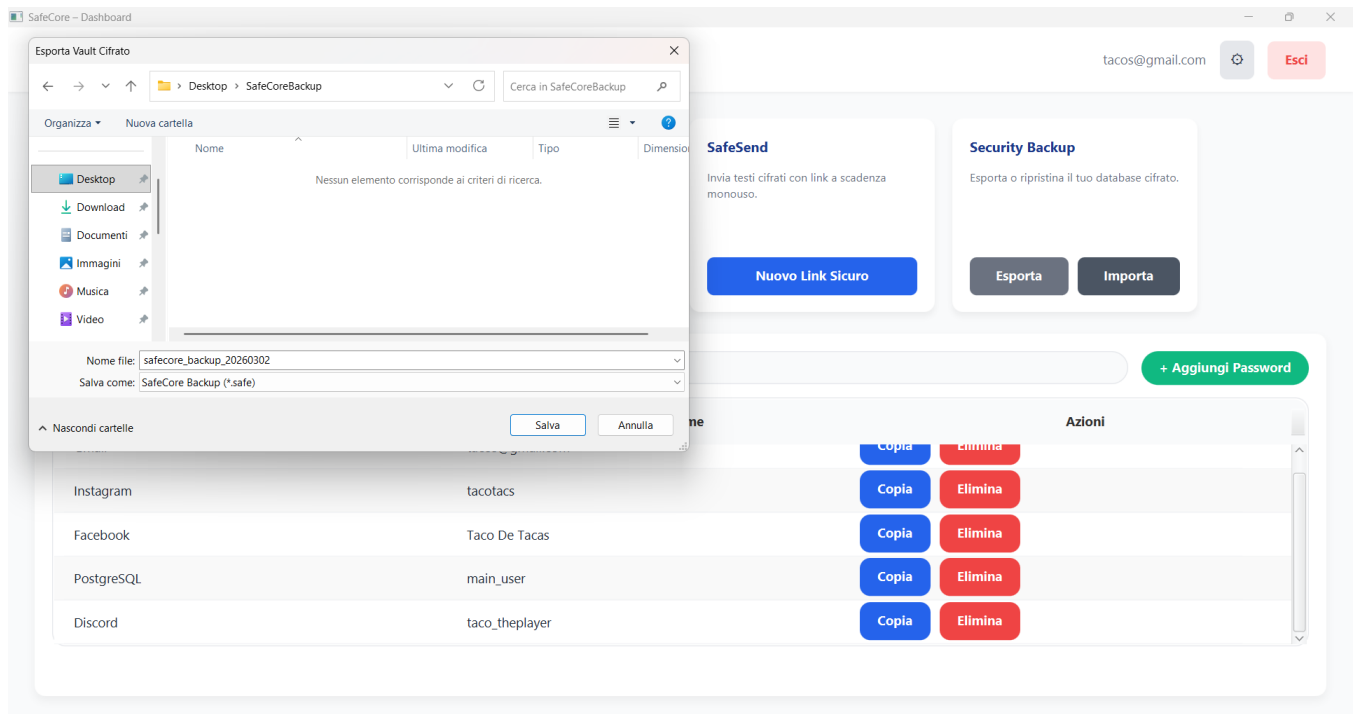
Mck. 3: Mockup della schermata principale del Vault.



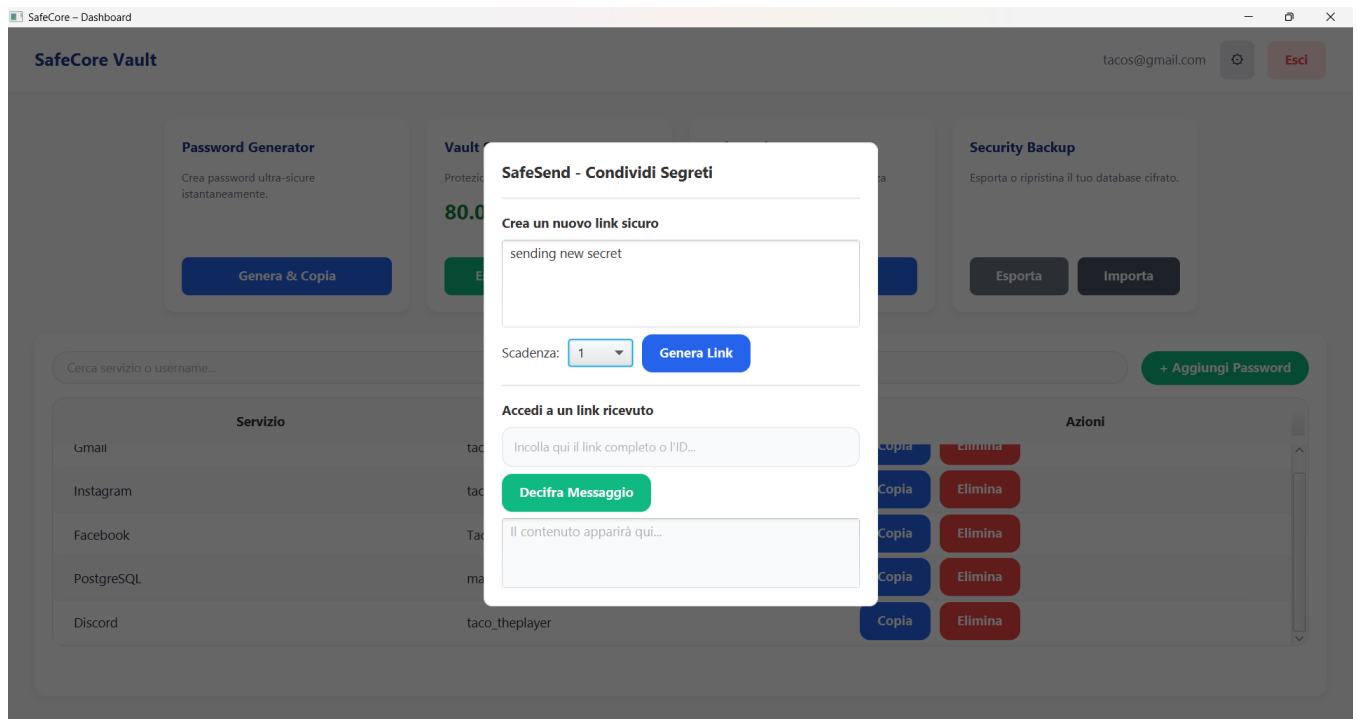
Mck. 4: Mockup dell'Inserimento Nuova Credenziale, UC-3.



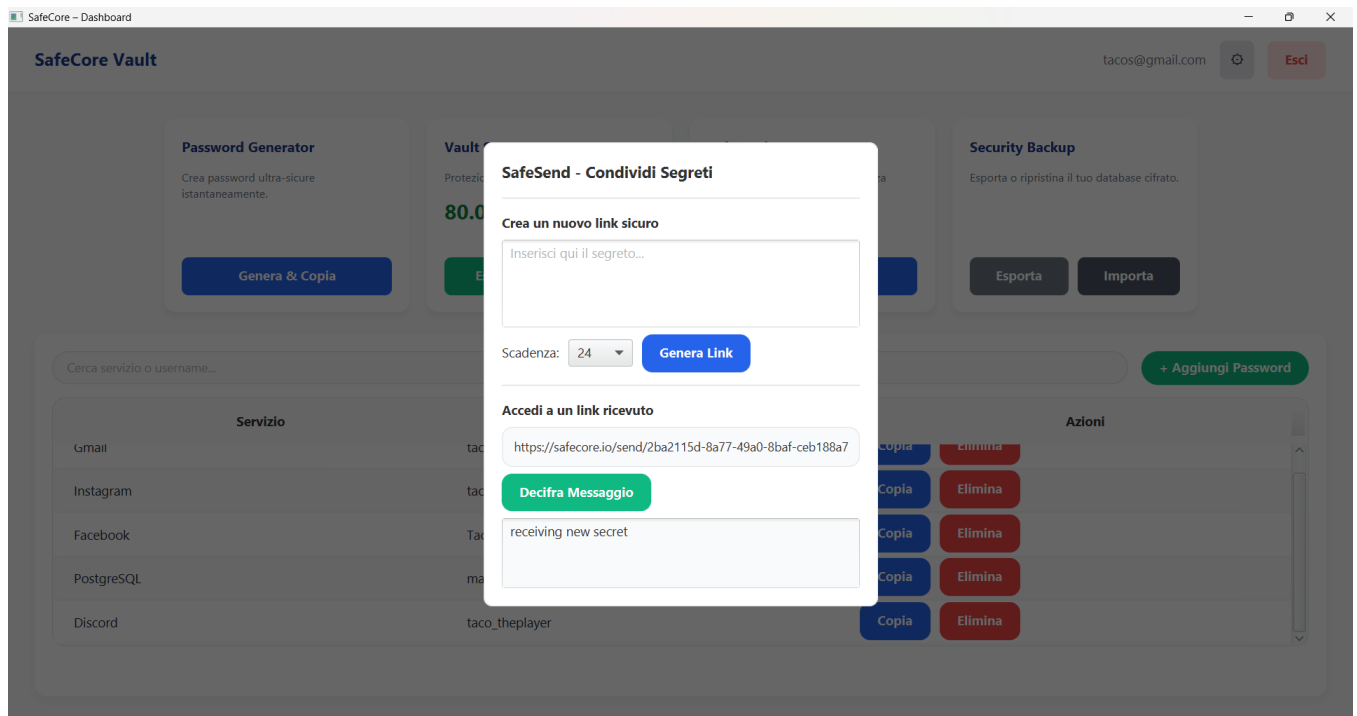
Mck. 5: Mockup della Generazione Password Sicura, UC-4.



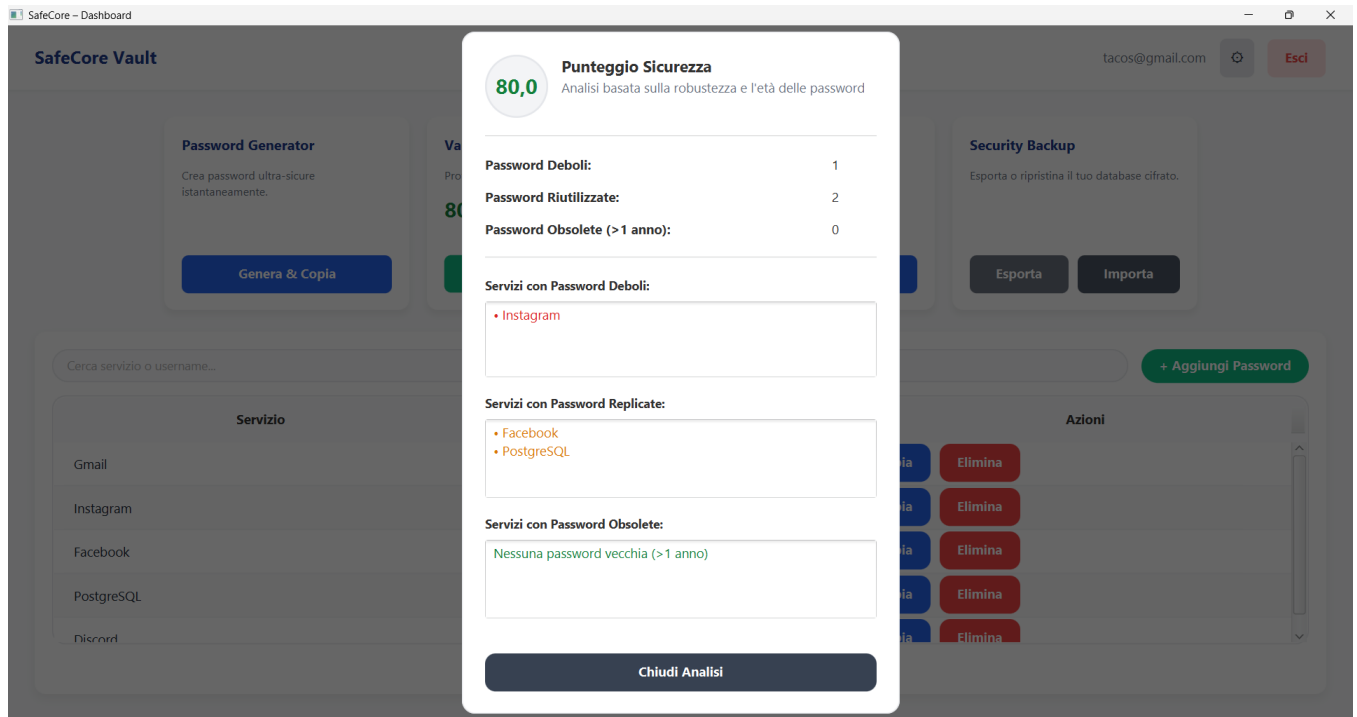
Mck. 6: Mockup dell'Esportazione Backup Vault, UC-5.



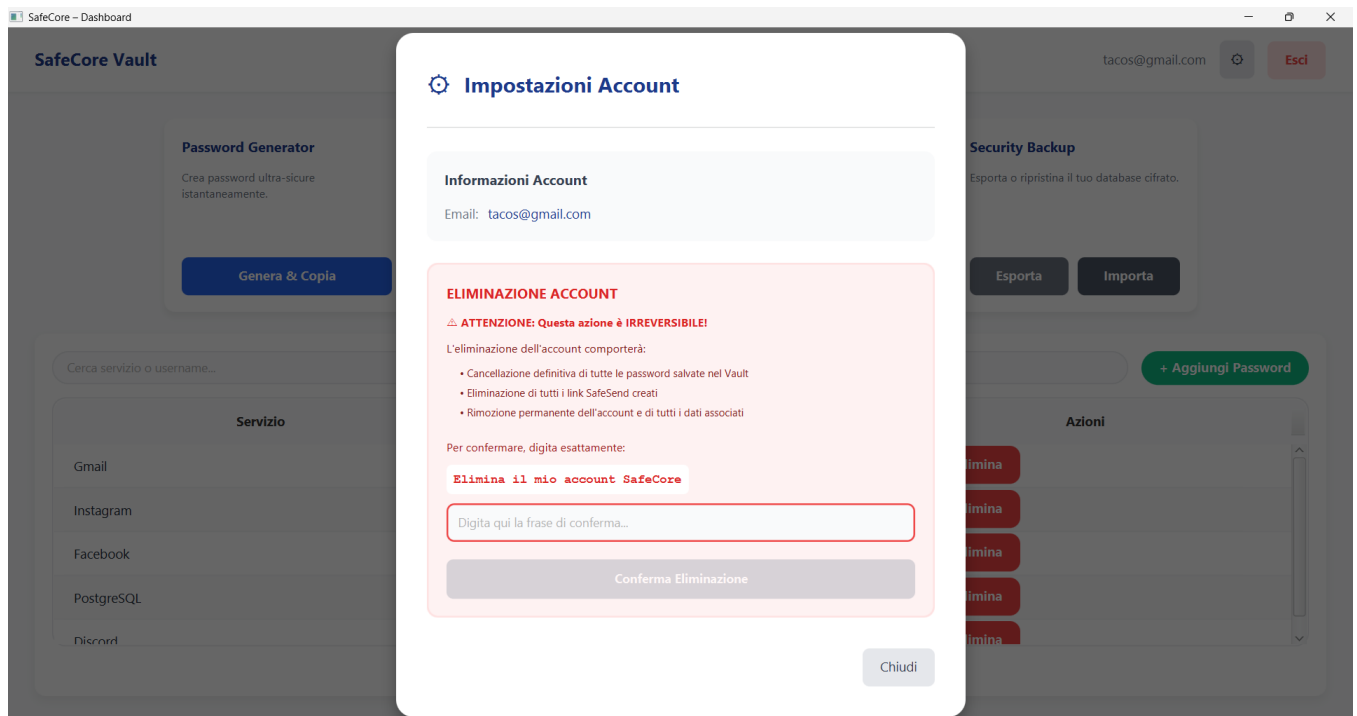
Mck. 7: Mockup dell'Invio di Segreto SafeSend.



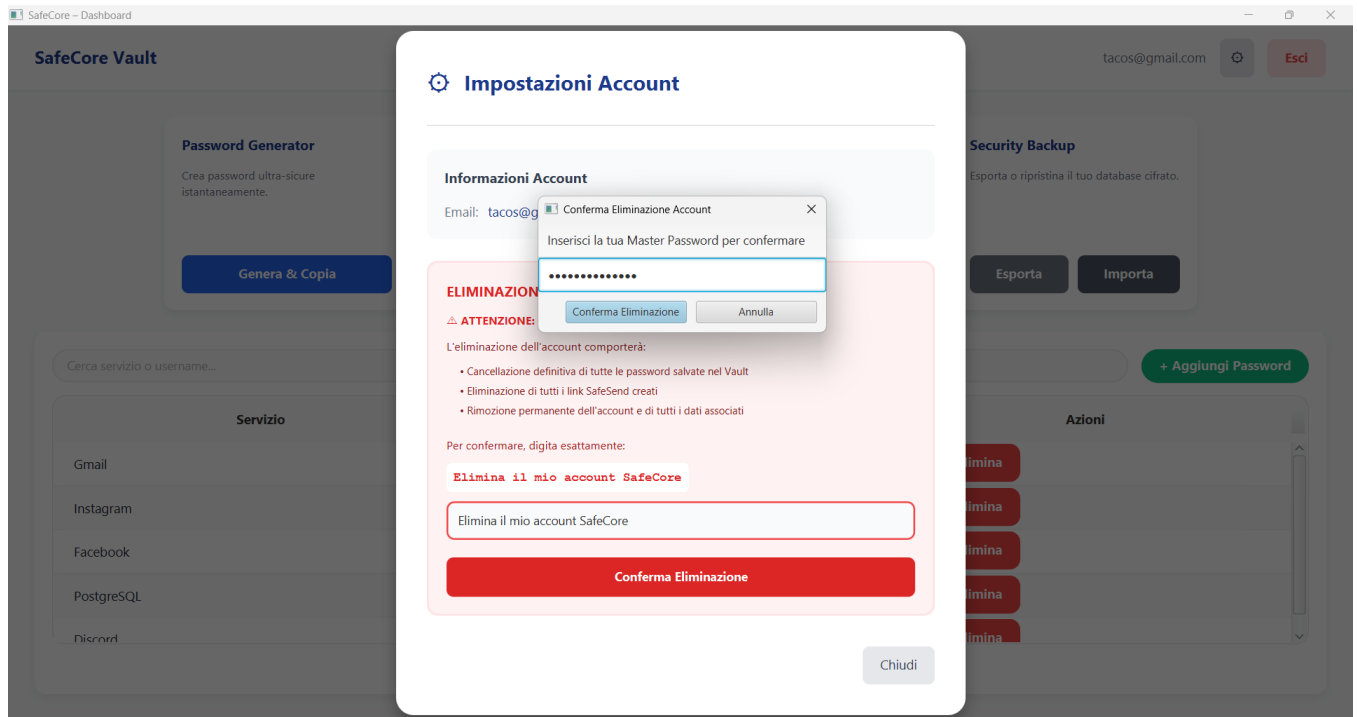
Mck. 8: Mockup dell'Accesso a Segreto SafeSend, UC-6.



Mck. 9: Mockup dell'Audit Sicurezza Vault, UC-7.



Mck. 10: Mockup della Richiesta di eliminazione account.



Mck. 11: Mockup dell'Eliminazione Account, UC-8

2.1.4 Requisiti non funzionali

Il design di SafeCore si rifà ai criteri di qualità definiti dallo standard ISO/IEC 25010, come illustrato nella Figura 2.2. Di seguito sono analizzati nel dettaglio.

- **Functional Suitability:** SafeCore implementa tutte le funzioni necessarie alla gestione sicura delle credenziali e alla condivisione temporanea (*SafeSend*), garantendo che i risultati delle operazioni crittografiche siano matematicamente accurati e rispondono alle specifiche di sicurezza dichiarate.
- **Reliability:** Garantisce Tolleranza ai guasti e Ripristinabilità. L'implementazione di transazioni atomiche per le operazioni di scrittura assicura che, in caso di eccezioni o arresti improvvisi, il database non subisca corruzioni, preservando lo stato consistente dei dati e garantendo la disponibilità del servizio anche in presenza di input non validi grazie a una robusta gestione delle eccezioni.
- **Performance Efficiency:** Focalizzata sul *Time-Behaviour*, l'integrazione di algoritmi crittografici ottimizzati e l'implementazione di un database PostgreSQL con ottimizzazione degli accessi ai dati tramite indici e query pre-compilate garantisce tempi di risposta costanti e rapidi, evitando colli di bottiglia durante le operazioni di sblocco del vault.
- **Operability:** Orientata all'Apprendibilità e alla Riconoscibilità dell'appropriatezza. L'interfaccia grafica separa chiaramente le funzioni di inserimento, ricerca e generazione password, guidando l'utente ed evitando input errati grazie a validatori integrati nei campi di testo.
- **Security:** Garantisce Riservatezza e Integrità dei dati. L'uso del meccanismo BCrypt per l'hashing della password principale assicura l'Autenticità dell'accesso, mentre la crittografia AES protegge i dati salvati nel database, impedendone la lettura in chiaro anche agli sviluppatori e ai gestori del database.
- **Compatibility:** Garantisce l'Interoperability e la Co-existence. L'architettura basata su standard aperti (JDBC, SQL) permette al software di scambiare dati in modo fluido con il database, garantendo allo stesso tempo la convivenza con altri processi di sistema senza conflitti nell'allocazione delle risorse.

- **Maintainability:** Rispetta la Modularity e l'Analyzability. Il codice è organizzato secondo un pattern architetturale scelto per separare la logica crittografica da quella di persistenza; questo facilita la Modificabilità in caso di aggiornamento degli algoritmi di sicurezza.
- **Portability:** Rispetta il criterio di Installability. L'uso del Java Runtime Environment e la configurazione del build system permettono di generare un archivio eseguibile unico, facilitando il trasferimento e l'esecuzione del software tra i sistemi Windows e macOS.

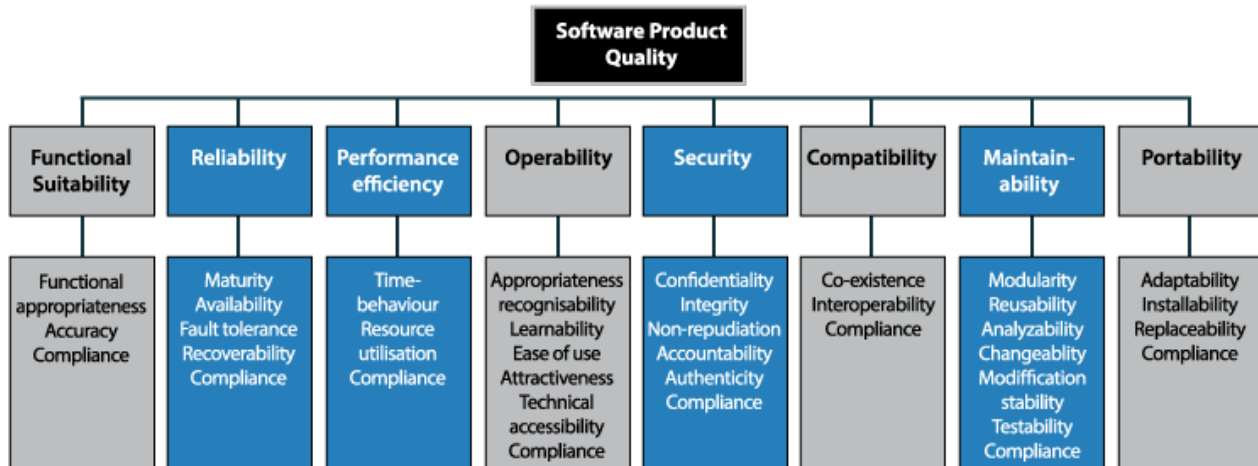


Figura 2.2: Caratteristiche della qualità del prodotto software (ISO/IEC 25010).

Approfondimento

Algoritmi di cifratura e hashing

L'obiettivo di SafeCore è quello di implementare sistemi di memorizzazione attraverso algoritmi di cifratura sicuri ed efficaci, come l'hashing della Master Password e la memorizzazione cifrata delle password nel vault. In questo progetto sono stati implementati i seguenti:

- **AES-256-CBC** (*Advanced Encryption Standard in modalità Cipher Block Chaining*)
È una configurazione completa di cifratura composta dall'algoritmo di cifratura **AES**, la modalità operativa **CBC** e il metodo **PKCS5** per il padding. AES è un algoritmo di cifratura simmetrica a blocchi con chiave a 256 bit, utilizzato come standard internazionale per cifratura di dati sensibili da banche, governi e sistemi di sicurezza informatica. Questo algoritmo lavora su blocchi di dimensione fissa 128 bit organizzati in matrici 4×4 attraverso 14 cicli detti *round*, in ognuno dei quali vengono applicate in sequenza operazioni matematiche di:
 - **SubBytes** Ogni byte viene sostituito in modo non lineare tramite una tabella chiamata S-Box, in modo da rendere complessa la relazione tra chiave e cifrato,
 - **ShftRows** Le righe della matrice vengono ruotate di un certo numero di posizioni per introdurre diffusione (nel senso di Shannon) dei dati,
 - **MixColumns** I byte di una colonna vengono mescolati tramite operazioni algebriche in un campo finito ($GF(2^8)$),
 - **AddRoundKey** Il risultato viene combinato tramite operazione di XOR ad una "sottochiave" derivata dalla chiave principale.

Prima di iniziare i round, viene applicato solo l'AddRoundKey con l'intera chiave, unico passaggio in cui l'intera chiave viene introdotta nel processo; nell'ultimo round invece non viene applicato il terzo passaggio. L'affidabilità di questo standard deriva dall'ottimo compromesso tra la

lunghezza della chiave, che offre 2^{256} possibili combinazioni, e l'elevata efficienza computazionale. La decifratura semplicemente applica le operazioni inverse in ordine inverso.

Dal momento che AES cifra solo blocchi di 16 byte, è necessaria una modalità di divisione di un messaggio più lungo in blocchi da 16 byte: lavorando in modalità CBC, il messaggio viene diviso in blocchi da 16 byte ed ogni blocco viene prima combinato tramite operazione di XOR col blocco precedente già cifrato, poi cifrato a sua volta. Per un blocco i , la formula di cifratura è scritta come

$$C_i = AES_K(P_i \oplus C_{i-1})$$

Il primo blocco viene combinato con un **IV (Initializing Vector)** casuale, il quale non si ripresenta due volte uguale quando utilizzato con la stessa chiave: questo garantisce che lo stesso messaggio cifrato due volte produca risultati diversi. Per recuperare il messaggio in chiaro l'algoritmo agisce in maniera inversa, partendo dall'ultimo blocco decifrato e combinandolo con il precedente, per il primo blocco di nuovo sarà usato l'IV.

Inoltre, poiché AES lavora su blocchi di 16 byte, l'ultimo blocco potrebbe non essere completo, perciò è previsto un meccanismo di completamento dei blocchi, il padding PKCS5. Questo meccanismo agisce in maniera molto semplice: se all'ultimo blocco mancano N byte per la dimensione di 16, si aggiungono i byte mancanti con valore N ciascuno. Così, ad esempio, se all'ultimo blocco mancano 4 byte, questo verrà riempito con quattro byte ciascuno di valore 04. In realtà, questo meccanismo prevede che, anche se l'ultimo blocco è completamente riempito, venga inserito un intero blocco finale con byte ciascuno di valore 16. Durante la decifratura, viene letto l'ultimo byte dell'ultimo blocco e, in base al suo valore, vengono eliminati gli ultimi byte, corrispondenti ai byte di padding.

- **PBKDF2** È una funzione derivatrice di chiavi basata su password ed è implementata per derivare la chiave crittografica per l'algoritmo AES a partire dalla Master Password. In particolare, questa funzione prende in input:
 - **Password**, la Master Password in chiaro, inserita dall'utente,
 - **Salt**, un valore casuale a 32 bit, unico per ogni utente, salvato in memoria,
 - **Iterazioni**, il numero di volte in cui ripetere l'algoritmo,
 - **Lunghezza**, la dimensione in bit che dovrà avere la chiave,
 - **Funzione di base**, tipicamente HMAC con SHA-256.

Questa funzione ha caratteristiche di sicurezza importanti in quanto permette di rallentare notevolmente e dunque interrompere attacchi *brute-force* e protegge da tabelle precalcolate grazie al valore casuale del salt.

Questo processo è svolto ad ogni login per decifrare le password presenti nel vault e renderle disponibili all'utente: infatti, con salt e password uguali si ottiene sempre la stessa chiave, utile dunque in caso di algoritmi di cifratura simmetrica come AES. Inoltre, funge anche come meccanismo di blocco del vault, che si sblocca solo quando viene inserita la master password corretta.

- **BCrypt** È un algoritmo di hashing adattivo irreversibile, standard de facto per la memorizzazione della Master Password nel database. Questo algoritmo, intenzionalmente lento, nasce con l'obiettivo di "complicare" il meccanismo di hashing e renderlo imprevedibile, rendendo inutile l'utilizzo di *Rainbow Tables* (tabelle precompilate di hashing comuni) per tentativi di *hacking*.

L'algoritmo si serve di due elementi, quali ancora un **salt** casuale di 128 bit e un **cost factor**, ossia un numero che indica la lentezza dell'operazione (ad es. se n è il *cost factor*, l'algoritmo opererà in 2^n iterazioni). Quest'ultimo ha l'utilità di rendere l'algoritmo mantenibile nel tempo: man mano che i computer aumentano di rapidità computazionale, sarà sufficiente modificare il *cost factor* per mantenere il processo adeguatamente lento.

L'algoritmo funziona così:

- **Inizializzazione** Viene generato il salt casuale e dato all'algoritmo insieme alla Master Password.
- **EksBlowfish** È il cuore dell'algoritmo. Blowfish è un cifrario che ha delle tabelle interne di numeri (chiamate *P-Array* per le permutazioni e *S-Boxes* per le sostituzioni) che vengono mescolate e sostituite usando proprio la password e il salt per un numero di volte pari a 2^c con c il *cost factor*. In pratica, la password e il salt vengono usati per costruire una "macchina cifrante" personalizzata in maniera lenta e costosa (*Eks* sta per *Expensive Key Schedule*).
- **Cifratura finale** La "macchina" creata viene poi utilizzata per cifrare per 64 volte una frase fissa, che nello standard è "*Orpheus beholder scry doubt*". Quest'ultimo passaggio genera il risultato finale.

Al termine, in memoria sarà salvata una stringa che contiene la versione dell'algoritmo, il fattore di costo, il salt e l'hash generato dall'algoritmo. Per la verifica, il sistema estrarrà fattore di costo e salt dalla stringa salvata nel database ed eseguirà di nuovo l'hashing, ottenendo un risultato che poi verrà confrontato con quello salvato e, a seconda della corrispondenza o meno, verrà consentito o no l'accesso al vault.

Capitolo 3

Progettazione e Implementazione

In questo capitolo viene analizzata la progettazione di SafeCore. Essa si è posta come obiettivo la garanzia di sicurezza crittografica, mantenibilità e scalabilità del Software.

3.1 Scelte progettuali e considerazioni architetturali

Le scelte principali che hanno guidato lo sviluppo includono:

- **Architettura Layered (4-Tier):** il sistema è stato suddiviso in quattro livelli indipendenti (*Presentation, Business Logic, Persistence, Security*) seguendo il principio di *Separation of Concerns*. Questa separazione permette di isolare la logica crittografica dai dettagli dell'interfaccia o del database. In questo modo, l'evoluzione tecnologica di un singolo modulo può avvenire senza impattare gli altri strati dell'applicazione.
- **Sicurezza Crittografica e Paradigma Zero-Knowledge:** fondamentale è stata l'adozione di un'architettura *Zero-Knowledge*. Il sistema è costruito per essere ignaro dei segreti dell'utente: la Master Password non viene mai memorizzata, ma trasformata in un hash non invertibile tramite BCrypt per la sola autenticazione. Le chiavi di cifratura AES-256 vengono derivate in memoria RAM esclusivamente durante la sessione e distrutte al logout. Questo garantisce che, anche in caso di accesso non autorizzato al database, i dati rimangano cifrati e del tutto inaccessibili.
- **Spring Boot, IoC e Dependency Injection:** l'integrazione di Spring Boot permette di gestire il ciclo di vita dei componenti in modo disaccoppiato. Grazie all'*Inversion of Control (IoC)* e alla *Dependency Injection*, i componenti ricevono le proprie dipendenze dall'esterno (tramite costruttore) anziché istanziarle internamente. Inoltre questo approccio rende il codice più modulare e permette di eseguire test in modo più facile.
- **DataBase PostgreSQL:** per la gestione dei dati è stato configurato un database **PostgreSQL** dedicato, per garantire persistenza e integrità in un ambiente di produzione reale. L'accesso al database è mediato dal *Repository Pattern*: questa astrazione è stata fondamentale per separare la logica applicativa dallo storage e ci ha permesso, in fase di test, di utilizzare database H2 in-memory.

3.2 Architettura del Sistema (Component Diagram)

L'architettura di SafeCore è stata progettata per essere modulare e disaccoppiata. Il **Component Diagram** (Figura 3.1) evidenzia la scomposizione del sistema in moduli indipendenti e il modo in cui questi interagiscono attraverso interfacce ben definite.

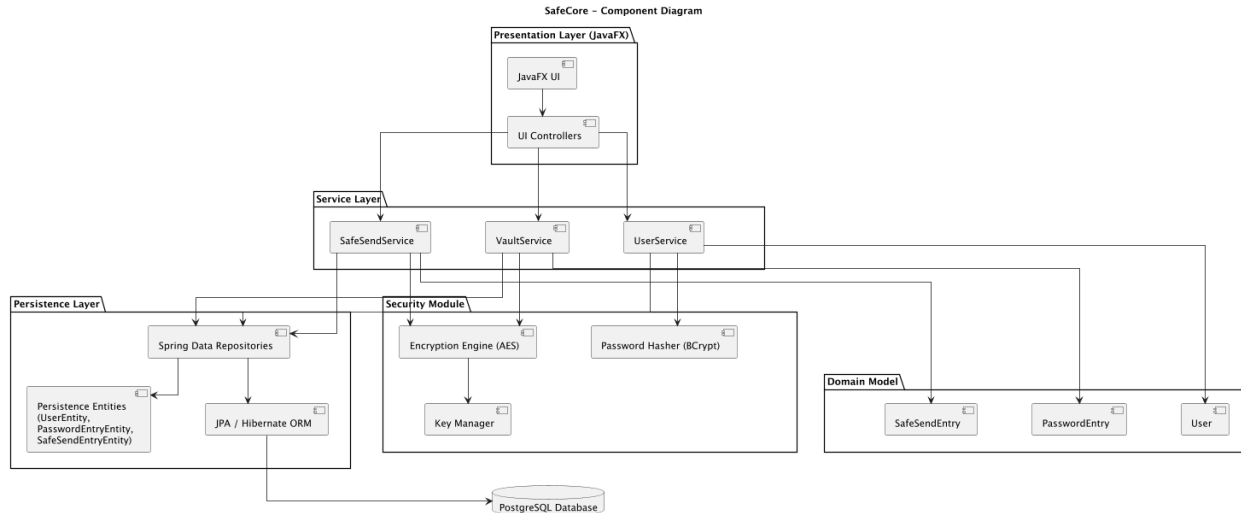


Figura 3.1: Diagramma dei Componenti

3.2.1 Analisi dei Macro-Componenti

Il sistema è formato da quattro macro-moduli, ciascuno con una responsabilità specifica all'interno del Software:

- **UI Component (Presentation):** realizzato con JavaFX e FXML, questo modulo gestisce esclusivamente la visualizzazione e l'acquisizione degli input. I *Controller* non contengono logica di business, ma delegano le operazioni ai servizi tramite interfacce, reagendo ai cambiamenti dello stato dell'applicazione.
- **Service Component (Business Logic):** rappresenta il centro operativo del sistema. Si occupa di coordinare le operazioni di gestione del vault, la validazione delle password e la gestione delle sessioni utente. Questo modulo fa da intermediario logico, tra il layer di presentazione e la persistenza dei dati, coordinando il flusso delle informazioni e l'applicazione delle regole di business.
- **Security Component:** questo modulo è un'entità isolata che espone funzionalità crittografiche. Viene invocato dai servizi per cifrare o decifrare i dati prima che questi vengano passati al layer di persistenza, garantendo che i dati in transito verso il database siano sempre protetti.
- **Persistence Component:** utilizzando Spring Data JPA, questo componente gestisce il dialogo con il database PostgreSQL. Astrae le query SQL complesse attraverso interfacce *Repository*, occupandosi della traduzione tra gli oggetti Java (*Entities*) e le tabelle relazionali.

3.2.2 Analisi del Component Diagram

Il Component Diagram (Figura 3.1) illustra una mappatura fedele con la struttura dei package e la logica implementativa:

- **Presentation Layer:** separa nettamente la parte grafica (*file .fxml*) dai controller (`com.safecore.ui.controller`, che gestiscono gli eventi dell'interfaccia iniettando i servizi necessari).
- **Service Layer:** svolge il ruolo di orchestratore centrale del sistema. Le interfacce `UserService`, `VaultService` e `SafeSendService` garantiscono il disaccoppiamento tra la logica di business e le implementazioni concrete.
- **Security Module:** entità distinta che racchiude l'integrità dei protocolli crittografici. Componenti come l'**Encryption Engine** e il **Key Manager** isolano la gestione delle chiavi, supportando visivamente il paradigma Zero-Knowledge.

- **Persistence Layer:** implementa il mapping verso il database **PostgreSQL** attraverso Spring Data JPA, utilizzando repository e classi *Entity* (*UserEntity*, *PasswordEntryEntity*) per astrarre la gestione dello storage.
- **Domain Model:** rappresenta le entità trasversali (*User*, *PasswordEntry*) che trasportano le informazioni attraverso tutti i livelli applicativi.

La struttura presenta un **flusso unidirezionale delle dipendenze**: l'interfaccia utente non comunica mai direttamente con il database o il modulo crittografico, ma deve necessariamente passare attraverso il Service Layer. Questa gerarchia rende valida l'architettura 4-Tier adottata, garantendo che ogni componente operi solo nel proprio ambito di competenza.

3.3 Gestione dei Dati (Database)

I dati del sistema sono stati gestiti attraverso un database relazionale sviluppato inizialmente in ambiente locale e successivamente migrato su infrastruttura serverless, per permettere accessibilità remota pur mantenendo la gestione in tempo reale sia attraverso l'istanza remota che attraverso l'ambiente di sviluppo locale.

La gestione integrata delle entità tramite JPA/Hibernate e delle connessioni con JDBC permette di servirsi di un database nel quale sono implementate tutte le operazioni CRUD (CREATE, READ, UPDATE, DELETE) gestite in maniera automatizzata.

3.3.1 Modellazione dei Dati (Schema ER)

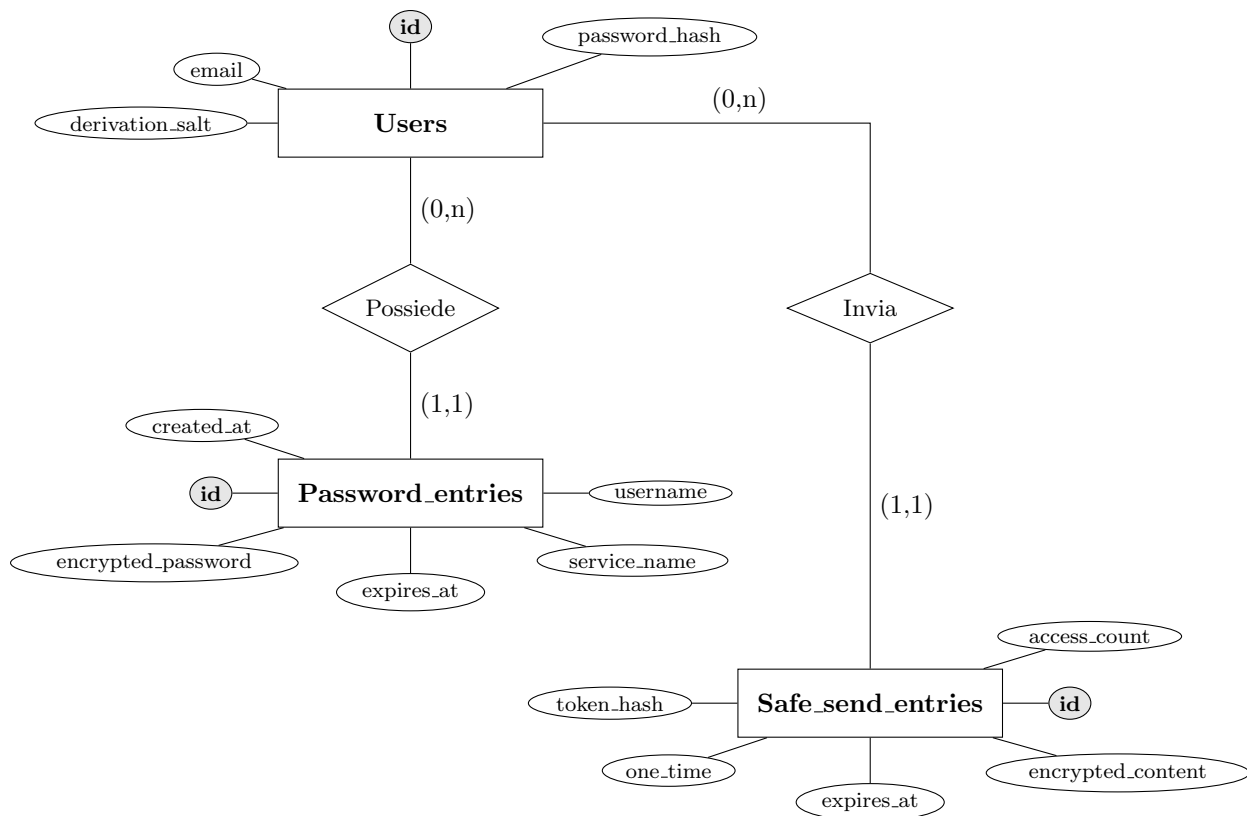


Figura 3.2: Schema ER per la modellazione dei dati nel database.

Il database è stato progettato seguendo un modello concettuale che pone l'entità **Users** al centro del sistema. Come illustrato in Figura 3.2, lo schema si articola su tre entità principali:

- **Users:** Rappresenta gli account degli utenti registrati. Ogni utente è identificato univocamente da un id e possiede attributi per l'autenticazione (email, password_hash) e la sicurezza (derivation_salt). Corrisponde alla classe `UserEntity`.
- **Password_entries:** Memorizza le credenziali salvate dall'utente. È un'entità legata all'utente, nella quale ogni voce deve appartenere necessariamente a uno e un solo account. Corrisponde alla classe `PasswordEntryEntity`.
- **Safe_send_entries:** Gestisce i contenuti inviati in modo sicuro. Anche in questo caso, ogni invio è tracciato e collegato all'utente mittente. Corrisponde alla classe `SafeSendEntity`.

Entrambe le relazioni tra le entità sono di tipo uno-a-molti: un singolo utente può possedere una o più password o non possederne nessuna e inviare uno o più messaggi sicuri o non inviarne alcuno, $(0, n)$. Di contro, ogni singola entry nelle tabelle dipendenti deve essere associata a uno e un solo utente $(1, 1)$, garantendo l'integrità referenziale e la separazione dei dati tra i diversi account.

3.3.2 Traduzione nel Modello Relazionale (PostgreSQL)

La transizione dallo schema ER alla sua implementazione fisica in PostgreSQL è avvenuta seguendo le regole di derivazione standard.

Users (id, derivation_salt, email, password_hash)

Password_entries (id, created_at, encrypted_password, expires_at, service_name, username, user_id)

user_id references Users(id)

Safe_send_entries (id, access_count, encrypted_content, expires_at, one_time, token_hash, user_id)

user_id references Users(id)

Le Chiavi Primarie (**PK**) degli schemi concettuali sono state implementate come tipi UUID, per garantire unicità certa anche in eventualità di distribuzione su scala maggiore, mentre per implementare le relazioni $(1, 1)$, $(0, n)$ sono stati inseriti i campi user_id nelle tabelle password_entries e safe_send_entries, che agiscono come *Foreign Key* (**FK**) per i vincoli di integrità referenziale sulla colonna id della tabella Users.

Inoltre, il database è impostato secondo un vincolo **ON DELETE CASCADE** per garantire che, in caso di eliminazione di un utente, tutti i dati sensibili ad esso associati vengano rimossi automaticamente, evitando record "orfani".

Per quanto riguarda la tipizzazione degli attributi, le password cifrate e il *salt* sono memorizzati come BYTEA, mentre le Master Password, le email e i token vengono memorizzati come VARCHAR(255), per ospitare gli hash cifrati. I timestamp (created_at, expires_at) utilizzano il tipo TIMESTAMP per gestire correttamente i fusi orari e l'attributo one_time è stato implementato come tipo BOOLEAN per ottimizzare lo spazio e la logica di controllo. Di seguito, un esempio di codice DDL (*Data Definition Language*) della creazione della tabella **users** generato ed eseguito da Hibernate durante l'inizializzazione dello schema. Dopo quel momento, viene rilevata la presenza della tabella nel database e semplicemente sincronizzata con eventuali modifiche all'entità `UserEntity`.

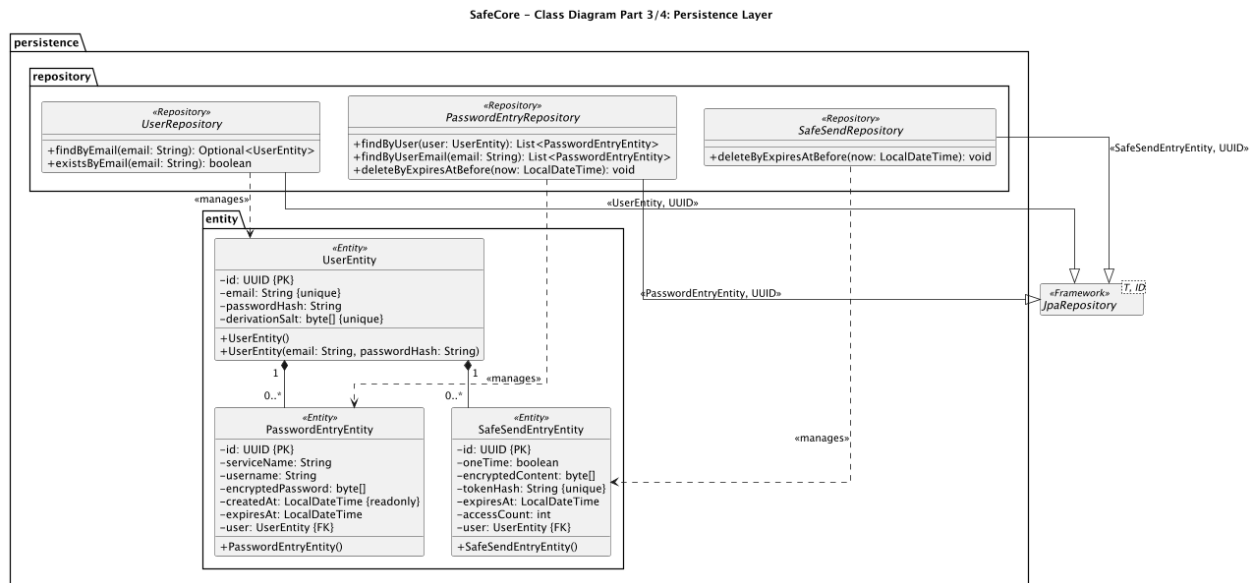
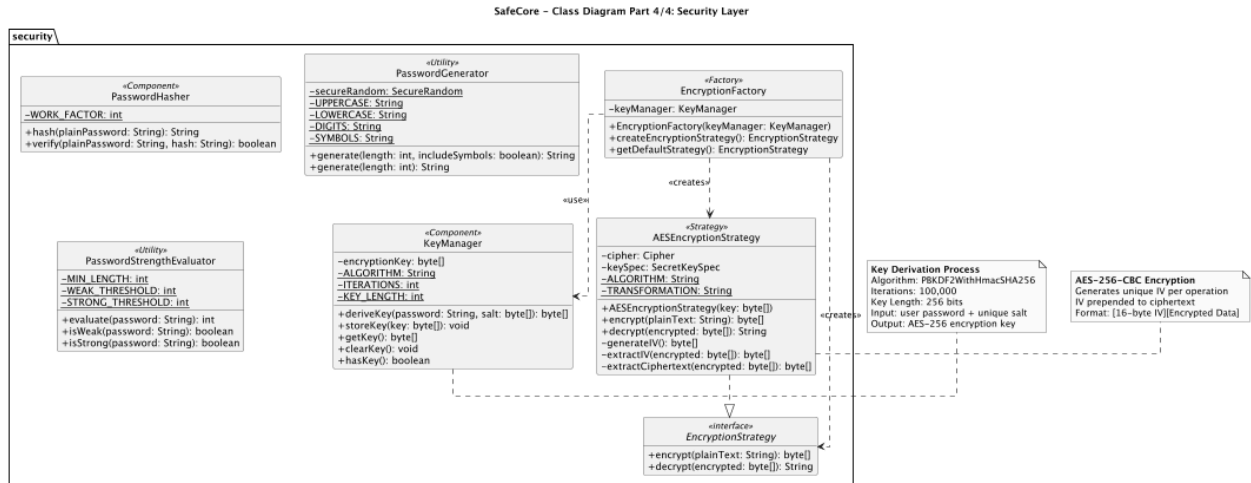
```

1 CREATE TABLE IF NOT EXISTS users (
2     id UUID PRIMARY KEY,
3     derivation_salt BYTEA UNIQUE NOT NULL,
4     email VARCHAR(255) UNIQUE NOT NULL,
5     password_hash VARCHAR(255) NOT NULL
6 );

```

Listing 3.1: Creazione della tabella **users** nel database.

Figura 3.4: Class Diagram - Parte 2: Livello di Business (Services, Domain, Hints, Exceptions).



La disposizione dei package e delle classi (Fig. 3.7) riflette una gerarchia a livelli in cui le dipendenze fluiscono uni-direzionalmente verso il basso (UI $\rightarrow$ Business $\rightarrow$ Security/Persistence):

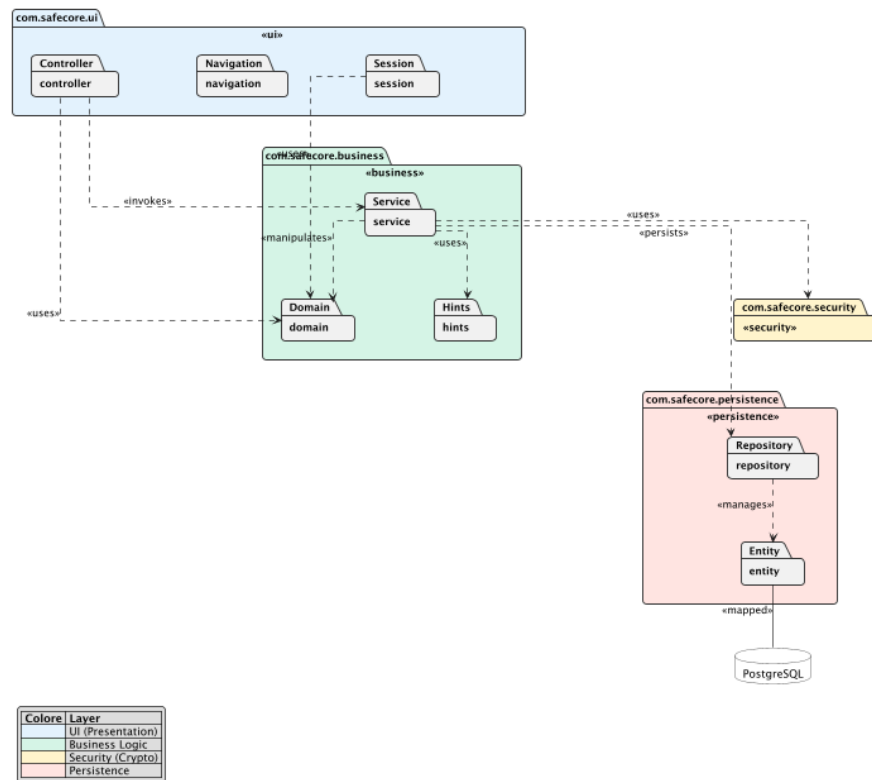


Figura 3.7: Package Diagram

*) **com.safecore.ui**: gestisce l'interfaccia JavaFX, la navigazione tra le viste e il contesto di sessione volatile.

*) **com.safecore.business**: contiene la logica di dominio, i contratti dei servizi, la gestione delle eccezioni custom e il sistema di validazione estensibile.

*) **com.safecore.security**: isola i componenti responsabili delle trasformazioni dei dati e della gestione delle chiavi, implementando pattern per la sostituibilità degli algoritmi.

*) **com.safecore.persistence**: astrae la memorizzazione fisica tramite repository, interfacciandosi con il database PostgreSQL in produzione.

3.5.1 Analisi delle Entità (Domain Model)

Le entità JPA, documentate nella (Fig. 3.6) del diagramma delle classi, modellano il centro informativo del sistema attraverso tre componenti principali. L'identità dell'utente è rappresentata dalla **UserEntity**, la quale memorizza l'identificatore univoco, le credenziali di accesso e i parametri necessari alla derivazione locale delle chiavi.

Il segreto cifrato viene invece modellato dalla **PasswordEntryEntity**, dove il campo `encryptedPassword` è appositamente progettato per incorporare l'Initialization Vector (IV) insieme al ciphertext; tale accorgimento garantisce l'unicità del dato salvato anche a fronte di input identici. Infine, la **SafeSendEntryEntity** si occupa di gestire la condivisione temporanea, supportando nativamente i vincoli di accesso monouso e le scadenze configurabili per i segreti condivisi.

3.6 Analisi Dettagliata dei Componenti per Package

L'architettura di SafeCore è stata progettata seguendo criteri di alta coesione e basso accoppiamento, distribuendo le responsabilità tra moduli logici definiti. Questa struttura non solo facilita la manutenzione, ma garantisce che i componenti critici per la sicurezza siano isolati dal resto dell'applicazione.

3.6.1 Package `com.safecore.ui` (Fig. 3.3)

Il livello di presentazione è responsabile della gestione dell'esperienza utente e del coordinamento dei flussi grafici. All'interno del sottopacchetto `ui.controller`, la logica di controllo è distribuita su otto classi che fungono da intermediari tra l'input dell'utente e i servizi di backend. Il `LoginController` e il `RegisterController` presidiano la fase di accesso, occupandosi rispettivamente della validazione delle credenziali e della guida alla creazione di profili sicuri. Una volta autenticati, il flusso è gestito dal `DashboardController`, che supervisiona il vault principale, supportato dall'`AddEntryController` per l'inserimento di nuovi segreti. Le funzionalità specializzate sono demandate all'`AuditController`, per l'analisi delle vulnerabilità, al `SafeSendController`, per la condivisione temporanea, e al `BackupController`, dedicato all'importazione ed esportazione dei dati. Infine, il `SettingsController` fornisce i mezzi per la gestione e la rimozione definitiva del profilo.

L'infrastruttura di supporto alla UI è garantita da quattro componenti essenziali. Lo `SceneNavigator` centralizza il passaggio tra le diverse schermate, mentre il `SessionContext` mantiene lo stato dell'utente e le relative chiavi in memoria volatile per tutta la durata dell'esecuzione. L'avvio del sistema è gestito dall'`AppLauncher`, mentre la stabilità operativa è assicurata dal `GlobalExceptionHandler`, che intercetta e neutralizza eventuali eccezioni impreviste traducendole in feedback comprensibili per l'utente.

3.6.2 Package `com.safecore.business` (Fig. 3.4)

Il centro applicativo risiede nel package `business`, dove vengono definite le regole di dominio e i processi logici. Il sottopacchetto `business.service` implementa la logica applicativa attraverso una serie di servizi e le relative implementazioni. Il binomio `UserService/Impl` e `VaultService` gestisce il ciclo di vita dell'identità digitale e dell'archivio password, garantendo che ogni operazione di modifica sia correttamente propagata. Parallelamente, il `SecurityAuditService` e il `SafeSendService` forniscono le logiche avanzate per la valutazione del rischio e la gestione dei token di condivisione, mentre il `BackupService` si occupa della portabilità dei dati attraverso formati cifrati.

L'integrità del sistema è supportata da un robusto apparato di gestione degli errori. Le classi di `Domain` rappresentano i modelli logici utilizzati nei calcoli, separati dalle entità di persistenza. La gerarchia delle `Exception` permette di identificare con precisione scenari critici, come la scadenza dei link o conflitti di registrazione. Infine, il modulo `Hints` implementa un motore a regole flessibile che analizza la robustezza delle password fornendo suggerimenti contestuali all'utente.

3.6.3 Package `com.safecore.security` (Fig. 3.5)

Questo modulo isola i componenti crittografici, assicurando che le operazioni sensibili avvengano in un ambiente protetto. L'`AESEncryptionStrategy` definisce le strategie di cifratura AES, mentre il `KeyManager` assume il ruolo critico di custode delle chiavi di sessione, occupandosi della loro derivazione e della distruzione sicura alla chiusura del programma. La protezione delle Master Password è affidata al `PasswordHasher`, che implementa algoritmi di hashing non invertibili. Completano il package diverse `Utilities` dedicate alla generazione di stringhe ad alta entropia e all'analisi statistica della robustezza dei segreti.

3.6.4 Package `com.safecore.persistence` (Fig. 3.6)

Il layer di persistenza gestisce la comunicazione con il database relazionale, assicurando che i dati siano memorizzati in modo coerente e sicuro. Nel sottopacchetto `persistence.entity`, le classi `UserEntity`, `PasswordEntryEntity` e `SafeSendEntryEntity` definiscono lo schema delle tabelle, stabilendo i vincoli di integrità e le relazioni tra gli utenti e i loro dati cifrati.

La manipolazione di queste informazioni è mediata dal sottopacchetto `persistence.repository`. Attraverso l'uso di interfacce specializzate come lo `UserRepository` per la gestione dei profili, il `PasswordEntryRepository` per le voci del vault e il `SafeSendRepository` per i segreti condivisi, il sistema astrae la complessità delle query SQL, fornendo metodi efficienti per il recupero dei dati, la ricerca per email e la pulizia automatica dei record scaduti.

3.6.5 Sintesi delle Responsabilità dei Componenti

La tabella seguente riassume le responsabilità dei 56 componenti distribuiti sui quattro layer architetturali:

Layer	Componente	Responsabilità
UI	Controller, Navigator, Session	Gestione interfaccia, navigazione e stato sessione.
Business	Services, Rules, Exceptions	Logica applicativa, validazione e audit di sicurezza.
Security	Strategies, KeyManager, Hasher	Cifratura, hashing e derivazione chiavi.
Persistence	Entities, Repositories	Mapping relazionale e persistenza su PostgreSQL.

Tabella 3.1: Responsabilità dei componenti di SafeCore suddivisi per layer.

3.7 Dinamica del sistema

Si può ora definire anche la struttura dinamica per analizzare come le componenti collaborino per eseguire le operazioni critiche del sistema. L'architettura a livelli di SafeCore impone un flusso verticale delle richieste: ogni operazione deve attraversare necessariamente il layer dei servizi e quello di sicurezza prima di interagire con la persistenza, garantendo che i protocolli crittografici non vengano mai bypassati.

3.7.1 Analisi dei flussi principali

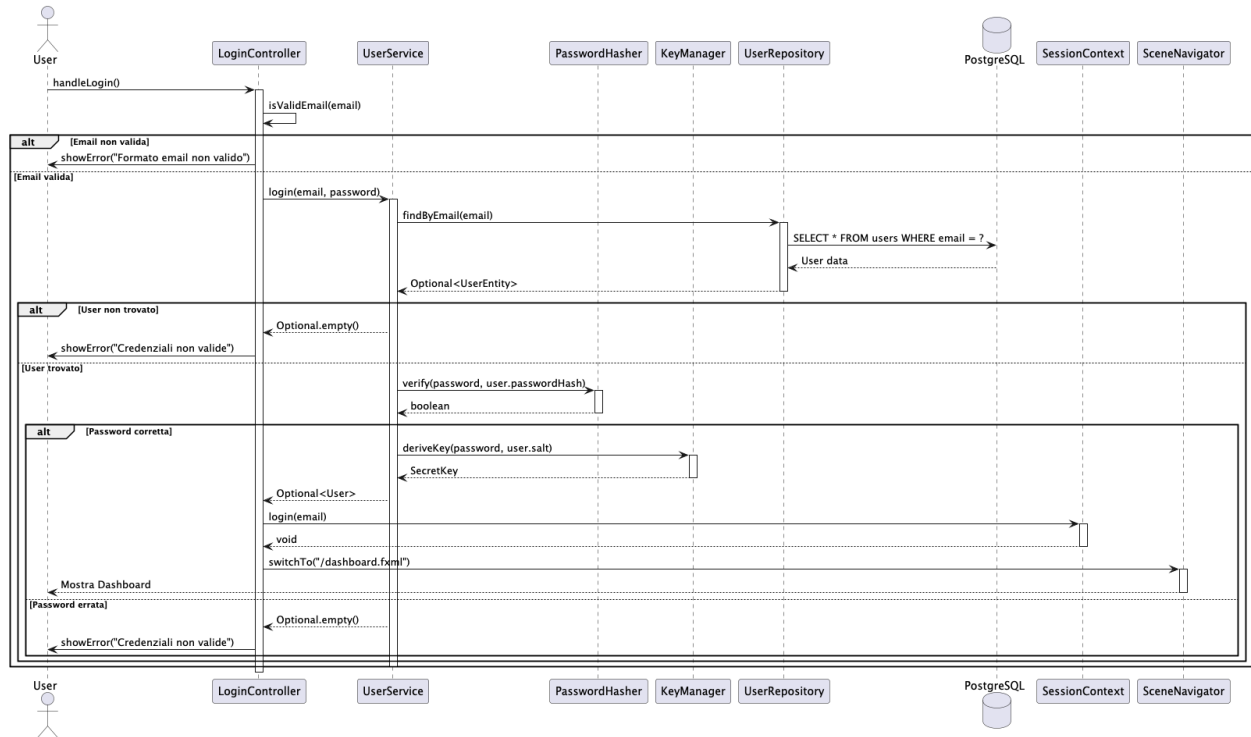


Figura 3.8: Diagramma di sequenza del processo di Login.

Il diagramma di Login (Fig. 3.8) mostra la gestione sicura dell'accesso. Una volta validato il formato dell'input, l'**UserService** interroga il database per recuperare i dati dell'utente: così il **PasswordHasher** verifica la corrispondenza dell'hash (BCrypt), mentre il **KeyManager** provvede alla derivazione della chiave simmetrica necessaria per l'intera sessione. Solo a seguito di queste verifiche, il sistema inizializza il contesto di sessione e permette il cambio di scena verso la dashboard.

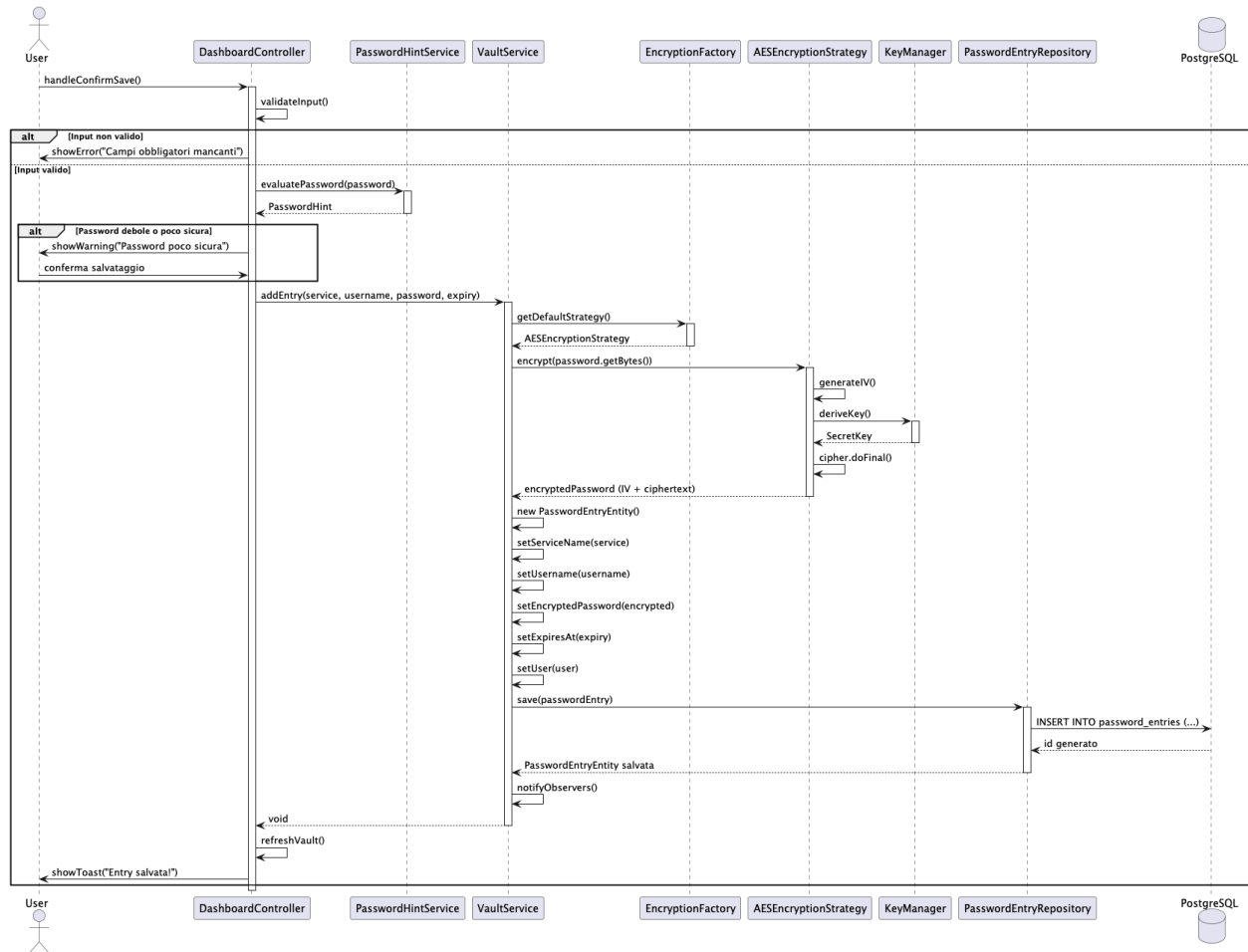


Figura 3.9: Diagramma di sequenza del salvataggio di una Password.

Il flusso di salvataggio di una nuova credenziale (Fig. 3.9) evidenzia invece l'integrità del processo di cifratura. Prima dell'archiviazione, il **VaultService** interagisce con la **EncryptionFactory** per ottenere la strategia crittografica corretta. Il testo in chiaro viene cifrato dal componente **AESEncryptionStrategy**, che utilizza il **KeyManager** per ottenere la chiave di sessione e genera un vettore di inizializzazione (IV) univoco. Solo il pacchetto cifrato risultante viene incapsulato in una **PasswordEntryEntity** e persistito su PostgreSQL, assicurando che sul database non transitino mai dati sensibili in chiaro.

3.7.2 Principi di Design

Per rendere il codice mantenibile, riusabile e robusto si sono seguiti dei principi di design basati sul concetto di **disaccoppiamento**.

Uno dei pilastri fondamentali è la **dipendenza da interfacce** piuttosto che da implementazioni concrete: un esempio è il caso di **UserService** e dell'interfaccia **EncryptionStrategy**, i componenti che utilizzano queste funzioni non sanno "come" il lavoro venga svolto, ma solo "cosa" aspettarsi. Questo significa che se in futuro si decidesse di cambiare il database o di adottare un nuovo algoritmo di cifratura, basterebbe creare una nuova implementazione dell'interfaccia senza dover modificare una singola riga di codice nei controller della UI.

Questo principio si sposa con l'**(Inversion of Control - IoC)**, facilitata dall'uso del framework Spring Boot. Infatti non è compito del programmatore dover istanziare manualmente ogni oggetto (comando *new*), ma è il sistema stesso a "iniettare" le dipendenze necessarie dove servono. Ad esempio, quando il **LoginController**

ha bisogno di verificare le credenziali, riceve automaticamente un'istanza pronta all'uso del servizio utente. Questo automatismo riduce la complessità del codice e facilita i test (vedi Cap. 4).

Altro aspetto cruciale è la **Separazione delle Responsabilità** (Separation of Concerns) per cui ogni package ha un compito preciso: la persistence si occupa solo di dialogare con il database, la security si isola per proteggere gli algoritmi e la business logic coordina il flusso senza preoccuparsi di come i dati verranno visualizzati. Un esempio pratico di questa separazione è l'uso di oggetti diversi per rappresentare gli stessi dati: le *Entity* sono ottimizzate per il database, mentre i modelli di *Domain* sono pensati per la logica applicativa. Questo evita che un cambiamento strutturale nelle tabelle del database si ripercuota su tutta l'applicazione.

Infine, il design è stato arricchito dall'uso di specifici design pattern.

3.8 Idiomi e Design Pattern

L'architettura di SafeCore integra i principali pattern dell'Ingegneria del Software per permettere di isolare le responsabilità e facilitare l'evoluzione del codice nel tempo.

3.8.1 Copia Difensiva (Defensive Copy)

Un aspetto critico nella gestione di dati sensibili in Java è la mutabilità degli array di byte. **Per garantire l'integrità delle informazioni crittografate all'interno del Domain Model**, è stato applicato l'idioma della **Copia Difensiva** nella classe `PasswordEntry`. Poiché un array è un oggetto mutabile, un riferimento esterno potrebbe modificarne il contenuto anche dopo la creazione dell'oggetto, violandone l'immutabilità e la sicurezza.

```
1 public final class PasswordEntry {
2     private final byte[] encryptedPassword;
3
4     private PasswordEntry(Builder builder) {
5         // Copia nel costruttore per evitare mutazioni esterne
6         this.encryptedPassword = builder.encryptedPassword != null
7             ? builder.encryptedPassword.clone() : null;
8     }
9
10    public byte[] getEncryptedPassword() {
11        // Copia nel getter per proteggere lo stato interno
12        return encryptedPassword != null ? encryptedPassword.clone() : null;
13    }
14 }
```

Listing 3.2: Esempio di Copia Difensiva in `PasswordEntry.java`

3.8.2 Strategy Pattern (Cifratura)

L'interfaccia `EncryptionStrategy` astrae l'algoritmo di cifratura, isolando la complessità della Java Cryptography Architecture dal resto dell'applicazione. Questa scelta permette di **rispettare il principio Open/-Closed**: il sistema è aperto all'introduzione di nuovi algoritmi crittografici ma chiuso a modifiche strutturali del livello di business.

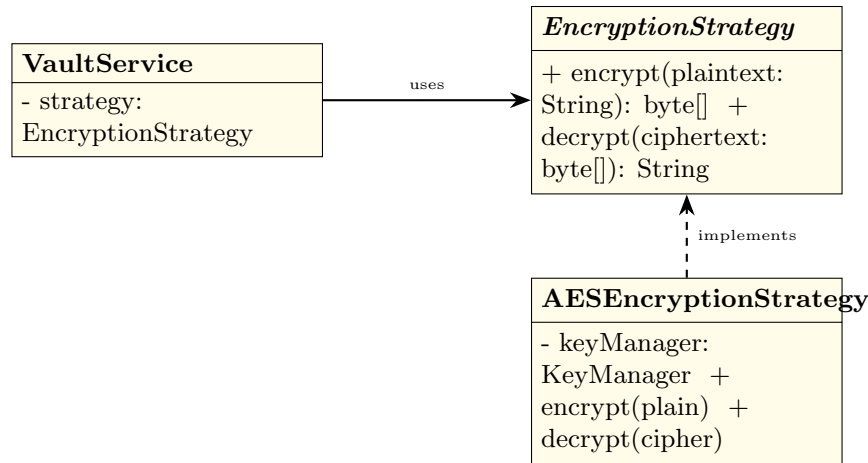


Figura 3.10: Pattern Strategy applicato alla cifratura dei dati.

L'implementazione concreta **AESEncryptionStrategy** utilizza l'algoritmo AES in modalità CBC con un vettore di inizializzazione (IV) generato casualmente per ogni operazione. Questo garantisce che testi in chiaro identici producano testi cifrati differenti, aumentando la resistenza del sistema alle analisi statistiche.

Il vantaggio principale dello **Strategy** consiste nel disaccoppiamento che consente di sostituire l'implementazione AES con algoritmi alternativi senza impattare sul **VaultService**. Inoltre in fase di testing, l'astrazione facilita l'uso di componenti mock per validare il flusso dei dati senza il peso computazionale della cifratura reale (vedi Cap. 4).

3.8.3 Observer Pattern (Sincronizzazione Reattiva)

Il **disaccoppiamento tra logica di business e UI** è gestito dal pattern **Observer**. L'idea è che il Service non deve sapere come è fatta la UI o quanti controller sono aperti. Quando il **VaultService** modifica i dati, si limita a notificare tutti gli osservatori.

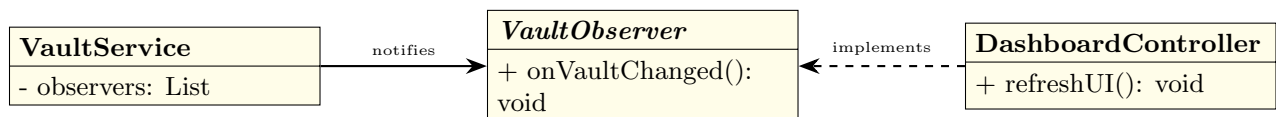


Figura 3.11: Pattern Observer: disaccoppiamento tramite inversione delle dipendenze.

In questo modo, componenti come il **DashboardController** possono aggiornare tabelle e grafici in tempo reale senza che il servizio debba invocarli direttamente, mantenendo i due livelli totalmente indipendenti.

```

1 // OBSERVER (Abstraction): Definisce il contratto per gli interessati.
2 // Il Subject dipende da questa interfaccia, non dai controller concreti.
3 public interface VaultObserver {
4     void onVaultChanged();
5 }
6
7 // SUBJECT (Concrete Subject): Gestisce lo stato e la lista degli osservatori.
8 @Service
9 @Service
10 public class VaultService {
11
12     //Il servizio conosce solo l'astrazione (DI)
13     private final List<VaultObserver> observers = new ArrayList<>();
14
15     public void addObserver(VaultObserver observer) {
  
```

```

16     observers.add(observer);
17 }
18
19 public void notifyObservers() {
20     //Notifica polimorfica: il Subject non sa chi risponde effettivamente
21     observers.forEach(VaultObserver::onVaultChanged);
22 }
23
24 @Transactional
25 public void addEntry(String service, String username, String plain) {
26     // ... logica salvataggio entry ...
27     notifyObservers(); // Notifica tutti gli osservatori
28 }
29 }
30
31 // CONCRETE OBSERVER: Implementazione specifica che reagisce alla notifica
32 @Component
33 public class DashboardController implements VaultObserver {
34     @Override //Metodo invocato dal Subject attraverso l'interfaccia
35     public void onVaultChanged() {
36         refreshVault(); // Aggiorna tabella UI
37         updateHealthScore(); // Ricalcola punteggio sicurezza
38     }
39 }

```

Listing 3.3: Pattern Observer: ruoli e inversione delle dipendenze

3.8.4 Builder e Repository

La gestione delle informazioni nel vault richiede il controllo sulla creazione degli oggetti e sulla loro archiviazione. A tale scopo, sono stati adottati i pattern **Builder** e **Repository per separare la struttura dei dati dalla logica di accesso al database**.

Il Builder viene impiegato per la classe `PasswordEntry` per garantire l'immutabilità dei segreti: permette di validare la presenza dei campi obbligatori prima di istanziare l'oggetto finale, prevenendo stati inconsistenti durante il passaggio tra i vari layer dell'applicazione.

```

1 public final class PasswordEntry {
2     private final UUID id;
3     private final String serviceName;
4     private final String username;
5     private final byte[] encryptedPassword;
6     private final LocalDateTime createdAt;
7
8     // Costruttore privato: l'oggetto puo essere creato solo tramite il Builder
9     private PasswordEntry(Builder builder) {
10         this.id = builder.id;
11         this.serviceName = builder.serviceName;
12         this.username = builder.username;
13         //COPIA DIFENSIVA
14         this.encryptedPassword = builder.encryptedPassword != null
15             ? builder.encryptedPassword.clone() : null;
16         this.createdAt = builder.createdAt;
17     }
18
19     public static class Builder {
20         private UUID id;
21         private String serviceName;
22         private String username;

```



```

23     private byte[] encryptedPassword;
24     private LocalDateTime createdAt;
25
26     public Builder id(UUID id) {
27         this.id = id;
28         return this;
29     }
30
31     public Builder serviceName(String sn) {
32         this.serviceName = sn;
33         return this;
34     }
35
36     public Builder username(String un) {
37         this.username = un;
38         return this;
39     }
40
41     public Builder encryptedPassword(byte[] ep) {
42         this.encryptedPassword = ep;
43         return this;
44     }
45
46     public Builder createdAt(LocalDateTime ca) {
47         this.createdAt = ca;
48         return this;
49     }
50
51     //VALIDAZIONE: verifichiamo i dati prima di istanziare l'oggetto
52     public PasswordEntry build() {
53         Objects.requireNonNull(serviceName, "Nome_servizio_obbligatorio");
54         Objects.requireNonNull(username, "Username_obbligatorio");
55         Objects.requireNonNull(encryptedPassword, "Password_cifrata_
           obbligatoria");
56         if (createdAt == null) this.createdAt = LocalDateTime.now();
57         return new PasswordEntry(this);
58     }
59 }
60 }

```

Listing 3.4: Builder per oggetti di dominio immutabili

Il pattern Repository astrae completamente l'accesso ai dati, permettendo alla logica di business di integrare con il database. Grazie all'integrazione con Spring Data JPA, il codice è indipendente dalla tecnologia specifica di database.

```

1  @Repository
2  public interface PasswordEntryRepository extends JpaRepository<PasswordEntryEntity,
      UUID> {
3      // Query derivata per il recupero del vault per utente
4      List<PasswordEntryEntity> findByUserEmail(String email);
5      List<PasswordEntryEntity> findByUser(UserEntity user);
6      // Gestione automatica della pulizia dei link scaduti
7      void deleteByExpiresAtBefore(LocalDateTime now);
8  }

```

Listing 3.5: Interfaccia Repository per l'astrazione della persistenza

3.8.5 Singleton e Factory

Per gestire le parti più delicate dell'app, come la sessione utente o la scelta degli algoritmi, sono stati usati i pattern **Singleton** e **Factory**.

Il pattern Singleton ci serve per la sessione: **garantisce che esista un'unica istanza del contesto di sessione** che contiene i dati dell'utente loggato. È un punto di riferimento globale e sicuro (anche se ci sono più processi attivi contemporaneamente).

```

1 public final class SessionContext {
2     private static volatile String loggedUserEmail;
3     private static LocalDateTime loginTime;
4
5     private SessionContext() {
6         // Costruttore privato impedisce istanziazione multiple
7     }
8
9     public static void login(String email) {
10         if (email == null || email.isBlank()) {
11             throw new IllegalArgumentException("Email_sessione_non_valida");
12         }
13         loggedUserEmail = email;
14         loginTime = LocalDateTime.now();
15     }
16
17     public static void logout() {
18         loggedUserEmail = null; //Reset completo stato
19         loginTime = null;
20     }
21
22     public static boolean isLoggedIn() {
23         return loggedUserEmail != null;
24     }
25
26     public static String getCurrentUserEmail() {
27         if (!isLoggedIn()) {
28             throw new IllegalStateException("Nessun_utente_loggato");
29         }
30         return loggedUserEmail;
31     }
32 }

```

Listing 3.6: Singleton per la gestione centralizzata della sessione

Infine, il pattern Factory **centralizza la logica di selezione degli algoritmi crittografici**. Invece di istanziare direttamente le classi di cifratura, il sistema interroga la **EncryptionFactory**, che mappa le strategie disponibili e restituisce quella corretta a runtime. Questo approccio semplifica l'estensione del software: per supportare un nuovo algoritmo è sufficiente aggiungere una nuova strategia senza modificare i servizi esistenti.

```

1 @Component
2 public class EncryptionFactory {
3     private final Map<String, EncryptionStrategy> strategies;
4
5     // Iniezione automatica di tutte le implementazioni di EncryptionStrategy
6     public EncryptionFactory(List<EncryptionStrategy> strategyList) {
7         this.strategies = strategyList.stream()
8             .collect(Collectors.toMap(
9                 s -> s.getClass().getSimpleName().replace("EncryptionStrategy", "").toUpperCase(),
10                 Function.identity()
11             ));

```

```
12 }
13
14 public EncryptionStrategy getStrategy(String type) {
15     EncryptionStrategy strategy = strategies.get(type.toUpperCase());
16     if (strategy == null) throw new IllegalArgumentException("Algoritmo non supportato"
17         );
18     return strategy;
19 }
```

Listing 3.7: Factory per la selezione dinamica degli algoritmi crittografici

Capitolo 4

Testing

Il testing è una fase fondamentale per garantire che ogni modulo rispetti le specifiche definite in fase di progettazione, analizzare i casi limite e gestire le eccezioni.

4.1 Strategia di Verifica

La metodologia a cui si ispira la verifica del sistema è la Piramide del Testing, che privilegia un'ampia base di test unitari veloci e isolati, salendo verso test di integrazione e di interfaccia più complessi. Questa struttura permette di individuare eventuali bug il più vicino possibile alla loro origine, riducendo drasticamente i costi di correzione.

In particolar modo, vengono integrati due approcci complementari per coprire ogni aspetto del software:

- **Test Strutturali (White-Box):** focalizzati sulla logica interna del codice e sull'analisi dei rami decisionali. Sono eseguiti principalmente a livello unitario per garantire la massima copertura (*Statement e Branch coverage*) delle componenti core.
- **Test Funzionali (Black-Box):** focalizzati sul comportamento esterno del sistema. Questi test verificano che per ogni input previsto nei **Casi d'Uso (UC)** corrisponda l'output atteso, ricalcando i flussi (principali e alternativi) definiti nei template, indipendentemente dall'implementazione interna.

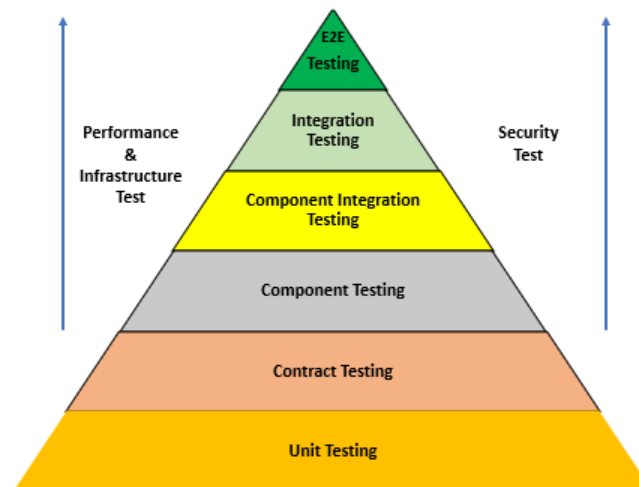


Figura 4.1: Piramide del Testing: alla base ci sono i test più numerosi, veloci ed economici. La sicurezza e le prestazioni vengono verificate come dimensioni trasversali all'intera struttura.

4.1.1 Tecnologie e Strumenti

L'ambiente di test è stato configurato attraverso più framework in modo da coprire ogni livello della piramide:

- **JUnit 5**: è il framework principale per l'esecuzione della suite di test.
- **Mockito**: utilizzato per il disaccoppiamento della Business Logic. Attraverso la creazione di oggetti simulati (Mocks), è possibile testare i servizi isolandoli dalle dipendenze reali (come il database o i moduli crittografici).
- **Database H2**: database in-memory utilizzato per i test di persistenza e integrazione. Sostituisce PostgreSQL in fase di test per garantire un ambiente temporaneo che si resetta automaticamente ad ogni esecuzione, evitando conflitti di dati tra i test.
- **TestFX**: il framework specifico per il testing delle interfacce grafiche JavaFX, permette di simulare interazioni utente reali (click, digitazione etc...) e verificare il comportamento dinamico della UI.

4.1.2 Configurazione e Annotazioni

L'ambiente di esecuzione è gestito tramite annotazioni standard che automatizzano l'iniezione delle dipendenze:

- **@ExtendWith(MockitoExtension.class)**: inizializza il supporto a Mockito, gestendo l'apertura e la chiusura dei mock.
- **@Mock**: definisce i collaboratori simulati (come i Repository) e permette di usare il costrutto "when(...).thenReturn(...)".
- **@InjectMocks**: inietta automaticamente i mock definiti all'interno della classe sotto test, rispettando il principio della *Dependency Injection*.
- **@BeforeEach**: metodo invocato prima di ogni singolo test per resettare i mock o inizializzare variabili; garantisce la totale indipendenza tra le esecuzioni.
- **@DirtiesContext**: utilizzato nei test d'integrazione e UI per segnalare a Spring di chiudere e ricreare il contesto applicativo.

4.2 Analisi dei Livelli di Testing

4.2.1 Livello 1: Domain Layer e Logica Pura (Unit Test)

Alla base della piramide si trovano i test unitari puri. L'obiettivo è verificare la correttezza matematica e algoritmica delle funzioni di sicurezza senza dipendenze esterne o framework pesanti. Questi test sono di tipo White-Box, poiché verificano direttamente l'integrità dei percorsi logici e degli stati interni degli algoritmi.

- **AESEncryptionStrategyTest**: Valida i cicli di cifratura/decifratura AES-256 e l'integrità del dato. È fondamentale per garantire che $\text{decrypt}(\text{encrypt}(x))=x$.
- **PasswordHasherTest**: Verifica che l'hashing (BCrypt) sia irreversibile e che la validazione delle password funzioni correttamente.
- **PasswordStrengthEvaluatorTest**: Analizza la logica di classificazione della robustezza (Weak, Medium, Strong) basata sull'entropia e sui set di caratteri.
- **PasswordGeneratorTest**: Controlla che le password generate rispettino i vincoli di lunghezza e casualità, agendo come test di conformità ai requisiti.

Esempio applicativo

L'algoritmo AES viene verificato utilizzando istanze reali. Il test è White-Box perché ispeziona la capacità del sistema di trasformare il dato e gestire stati non validi.

```

1  private EncryptionStrategy strategy;
2  private KeyManager keyManager;
3
4  @BeforeEach
5  void setUp() {
6      // Inizializzazione centralizzata per ogni test
7      keyManager = new KeyManager();
8      keyManager.initialize("MasterPassword123!", new byte[32]);
9      strategy = new AESEncryptionStrategy(keyManager);
10 }
11
12 @Test
13 void testEncryptDecryptSuccess() {
14     // ARRANGE
15     String originalText = "SecretData123!";
16
17     // ACT: Esecuzione del ciclo di cifratura e decifratura
18     byte[] encrypted = strategy.encrypt(originalText);
19     String decrypted = strategy.decrypt(encrypted);
20
21     // ASSERT: Verifica delle proprietà crittografiche (White-Box)
22     assertNotNull(encrypted);
23     assertNotEquals(originalText, new String(encrypted)); // Testo oscurato
24     assertEquals(originalText, decrypted);                 // Integrità del dato
25 }
26
27 @Test
28 void testEncryptNullThrows() {
29     // Boundary Testing: l'input nullo deve sollevare un'eccezione
30     assertThrows(IllegalArgumentException.class, () -> strategy.encrypt(null));
31 }

```

Listing 4.1: Validazione algoritmica della cifratura AES con Setup centralizzato

Data la natura deterministica dell'algoritmo AES, l'utilizzo di oggetti mock risulterebbe insensato e quindi si predilige l'uso di classi reali per garantire che la logica di cifratura sia priva di effetti collaterali. Il test convalida la proprietà fondamentale di invertibilità: $\text{decrypt}(\text{encrypt}(x))=x$. Un eventuale fallimento circoscriverebbe immediatamente l'errore alla logica algoritmica.

Inoltre, attraverso il Boundary Testing (es. input nulli o malformati), si assicura che il sistema intercetti le anomalie in modo esplicito, prevenendo la propagazione di errori silenti. Infine, l'elevata efficienza computazionale di questi test (eseguiti nell'ordine dei millisecondi) garantisce un feedback immediato nel ciclo di integrazione continua.

4.2.2 Livello 2: Business Logic e Service Layer (Mock Testing)

Il secondo livello si concentra sui Service, che coordinano la logica applicativa. In questa fase, l'approccio è di tipo **ibrido**: si adotta una metodologia **Black-Box** per convalidare l'output funzionale (risultati e gestione delle eccezioni) e una componente **White-Box** attraverso l'uso di **Mockito**. Quest'ultimo permette di ispezionare il comportamento interno, verificando che le interazioni tra i componenti avvengano come prestabilito.

Esempi di classi testate in questo livello:

- **UserServiceTest**: Gestisce la registrazione, il login utente e l'eliminazione account. (collegato a UC4 - Eliminazione Account).

- **VaultServiceTest**: Valida le operazioni CRUD, assicurandosi che i dati vengano passati alla strategia di cifratura prima di essere inviati al repository.
- **SafeSendServiceTest**: Gestisce la logica dei link "burn-after-reading" e le eccezioni personalizzate (`LinkNotFoundException`, `ExpiredLinkException`). (collegato a **UC5 - Condivisione Sicura**).
- **SecurityAuditServiceTest**: Verifica l'algoritmo di calcolo dello *Health Score* del vault. (collegato a **UC7 - Security Audit**).
- **PasswordHintServiceTest**: Testa la generazione di suggerimenti dinamici basati su regole (`MinLengthRule`, `ComplexityRule`).

Esempio applicativo Un test significativo riguarda il metodo `secureDeleteAccount`. Qui l'approccio White-Box è fondamentale poiché non ci si può limitare a verificare che l'utente sparisca dal database, ma bisogna garantire che i dati sensibili subiscano una "bonifica" prima dell'eliminazione fisica per impedirne il recupero.

```

1      @Test
2      void secureDeleteAccount_withPasswordEntries_sanitizesAndDeletesThem() {
3          // ARRANGE: Preparazione del contesto e dell'utente
4          String email = "user@test.com";
5          UserEntity user = new UserEntity();
6          user.setId(UUID.randomUUID());
7          user.setEmail(email);
8          user.setPasswordHash("hashed_password");
9
10         // Simulazione di entry esistenti nel vault
11         PasswordEntryEntity entry1 = new PasswordEntryEntity();
12         entry1.setEncryptedPassword(new byte[]{1, 2, 3});
13         List<PasswordEntryEntity> entries = Arrays.asList(entry1);
14
15         // Configurazione dei mock (Isolamento dei componenti esterni)
16         when(userRepository.findByEmail(email)).thenReturn(Optional.of(user));
17         when(passwordHasher.verify(anyString(), anyString())).thenReturn(true);
18         when(passwordEntryRepository.findByUserEmail(email)).thenReturn(entries);
19
20         // ACT: Esecuzione della logica di business
21         service.secureDeleteAccount(email, "Password123!");
22
23         // ASSERT: Verifica White-Box delle interazioni e della sicurezza
24         verify(passwordEntryRepository).saveAll(anyList()); // Bonifica (overwrite)
25         verify(passwordEntryRepository, times(2)).flush(); // Persistenza
26                                     intermedia
27         verify(passwordEntryRepository).deleteAll(entries); // Eliminazione fisica
28         verify(userRepository).delete(user); // Rimozione utente
29         verify(keyManager).clear(); // Pulizia chiavi in
                                     RAM
30     }

```

Listing 4.2: Test di `secureDeleteAccount` in `UserServiceTest` con Mockito

L'applicazione del pattern **AAA (Arrange-Act-Assert)** combinata con i metodi `verify()` di Mockito certifica che il servizio rispetti il protocollo di sicurezza: l'invocazione della bonifica (`saveAll`) deve precedere la cancellazione (`deleteAll`). L'istruzione `when().thenReturn()` simula il database e il validatore, garantendo l'esecuzione del flusso principale in totale isolamento. Infine, la verifica su `keyManager.clear()` garantisce la prevenzione di *data leakage* nella memoria RAM al termine del ciclo di vita dell'account.

4.2.3 Livello 3: Persistence Layer e Testing dei DAO

In questa fase si verifica la reale comunicazione tra l'applicazione e lo strato di persistenza. Per coniugare velocità d'esecuzione e indipendenza dall'ambiente di produzione, è stato adottato il **database in-memory H2**.

L'utilizzo di H2 garantisce uno stato di partenza sempre pulito grazie alla proprietà *create-drop*. In questo livello si eseguono test sui **DAO** per validare query JPQL custom, vincoli di integrità e il corretto funzionamento delle entità JPA tramite l'uso di `@DataJpaTest` e `TestEntityManager`.

- **UserRepositoryTest**: verifica i vincoli di unicità sull'indirizzo email e le operazioni di recupero tramite ID.
- **SafeSendRepositoryTest**: assicura la persistenza dei dati condivisi e il corretto funzionamento delle query di pulizia per i link scaduti.

4.2.4 Livello 4: Test Funzionali e di Integrazione

I test di integrazione rappresentano una verifica di tipo **Black-Box** volta a confermare la coerenza dell'intero sistema. A differenza dei test unitari, qui vengono utilizzate le istanze reali di ogni servizio per validare i flussi definiti nei **Casi d'Uso (UC)**.

Matrice di Tracciabilità Test-Use Cases

Ogni classe di test è stata progettata per coprire uno o più passi dei flussi principali e alternativi descritti nel Capitolo 2:

UC	Titolo	Stato Test	File di Riferimento
UC1	Registrazione	Completo	<code>UserServiceTest.java</code> , <code>UserRepositoryTest.java</code>
UC2	Login e Sblocco	Completo	<code>UserServiceTest.java</code> , <code>SafeCoreIntegTest.java</code>
UC3	Nuova Credenziale	Completo	<code>VaultServiceTest.java</code> , <code>PasswordServiceTest.java</code>
UC4	Generatore PW	Completo	<code>PasswordGeneratorTest.java</code>
UC5	Backup	Completo	<code>BackupIntegrationTest.java</code>
UC6	SafeSend	Eccellente	<code>SafeSendServiceTest.java</code> , <code>SafeSendRepositoryTest.java</code>
UC7	Security Audit	Completo	<code>SecurityAuditServiceTest.java</code>
UC8	Eliminazione	Completo	<code>UserServiceTest.java</code> (metodo <code>secureDeleteAccount</code>)

Tabella 4.1: Matrice di tracciabilità tra Casi d'Uso e suite di test.

Esempio applicativo di test di integrazione End-to-End Il `SafeCoreIntegrationTest` dimostra come tutti i livelli dell'architettura collaborino correttamente, dalla registrazione alla decifratura dei dati persistiti.

```

1 @SpringBootTest
2 @Transactional
3 class SafeCoreIntegrationTest {
4
5     @Autowired
6     private UserService userService;
7
8     @Autowired
9     private VaultService vaultService;
10
11     @Test
12     void testFullWorkflow() {

```

```

13     String email = "integration@test.com";
14     String password = "SecurePassword123!";
15
16     // 1. Registrazione
17     User user = userService.register(email, password);
18     assertNotNull(user);
19     assertEquals(email, user.getEmail());
20
21     // 2. Login
22     Optional<User> loggedIn = userService.login(email, password);
23     assertTrue(loggedIn.isPresent());
24
25     // Simuliamo la sessione
26     SessionContext.login(email);
27
28     try {
29         // 3. Aggiunta Segreto
30         vaultService.addEntry("TestService", "testUser", "secret123");
31
32         // 4. Verifica
33         List<PasswordEntryEntity> entries = vaultService.
34             getEntriesForCurrentUser();
35         assertFalse(entries.isEmpty());
36         assertEquals("TestService", entries.get(0).getServiceName());
37
38         String decrypted = vaultService.decryptPassword(entries.get(0).
39             getEncryptedPassword());
40         assertEquals("secret123", decrypted);
41     } finally {
42         SessionContext.logout();
43     }
44 }

```

Listing 4.3: Test di integrazione del flusso principale (Rif. UC1)

Test di Performance e Requisiti Non Funzionali

Come richiesto per garantire un'esperienza utente fluida, sono stati implementati test di performance per monitorare il costo computazionale delle operazioni di sicurezza. In particolare, si è verificato che l'hashing BCrypt (cost factor 12) e la cifratura AES rientrino in parametri accettabili.

```

1     @Test
2     @DisplayName("Encrypt operation should complete within reasonable time")
3     void encryptPerformance_shouldCompleteWithinTimeLimit() {
4         String plainText = "This is a test password to encrypt";
5
6         long startTime = System.nanoTime();
7         byte[] encrypted = encryptionStrategy.encrypt(plainText);
8         long endTime = System.nanoTime();
9
10        long durationMicros = (endTime - startTime) / 1_000;
11
12        // AES e' molto veloce, dovrebbe completare in meno di 10ms
13        assertTrue(durationMicros < 10_000,
14            "Encrypt operation took too long: " + durationMicros + "us(
15                expected < 10000us)");
16    }

```

Listing 4.4: Test di Performance per la cifratura AES

4.2.5 Livello 5: Interfaccia Utente e TestFX

L'apice della piramide è rappresentato dai test UI eseguiti con **TestFX**. Questi test assicurano che l'interazione umana (click, digitazione) produca il corretto feedback visivo e che la logica di business sia collegata correttamente ai controller JavaFX.

Le principali suite di test UI includono:

- **LoginControllerTest**: Verifica il flusso di autenticazione, validando che i messaggi di errore (es. "Credenziali non valide") appaiano correttamente e che i campi siano obbligatori (Rif. UC2).
- **RegisterControllerTest**: Valida la creazione dell'account, testando in particolare la sincronizzazione tra i campi password e la funzionalità di generazione automatica di password sicure (Rif. UC1, UC4).
- **SafeSendControllerTest**: Assicura che l'interfaccia di condivisione sicura gestisca correttamente gli input dell'utente, le opzioni di scadenza e la visualizzazione del link generato (Rif. UC6).
- **SettingsControllerTest**: Verifica le operazioni critiche del profilo, come l'eliminazione sicura dell'account, che richiede l'inserimento di una frase di conferma specifica per abilitare il pulsante di distruzione dati (Rif. UC8).

L'uso di `WaitForAsyncUtils` garantisce la sincronizzazione tra il thread dei test e il thread grafico di JavaFX.

4.3 Analisi della Copertura e Risultati Finali

Per misurare l'efficacia della suite di test del progetto SafeCore, è stato utilizzato lo strumento **JaCoCo** (Java Code Coverage). L'analisi si è focalizzata soprattutto sulla copertura dei rami decisionali (**Branch Coverage**), parametro rigoroso per determinare la reale qualità dei test in un software di sicurezza.

4.3.1 Statistiche della Suite di Test

Il processo di verifica ha coinvolto l'intero stack applicativo, organizzando circa 135 test distribuiti su 28 file. Il risultato finale attesta una percentuale di successo del 100% dei test.

Categoria	File Test	Test Totali	Esito	Copertura Linee
Security & Crypto	6	19+	PASS	96.8%
Business Logic	7	45+	PASS	90.3%
Persistence Layer	2	15+	PASS	88.5%
UI Layer (TestFX)	4	39	PASS	83.4%
Exception Handling	7	14+	PASS	100.0%
TOTALE	28	135	100%	88.2%

Tabella 4.2: Riepilogo generale della suite di test e distribuzione della copertura.

Risultati importanti sono quelli provenienti dai test del **Package Security**. Le classi crittografiche fondamentali, come `AESEncryptionStrategy` e `PasswordHasher`, hanno raggiunto una copertura totale (100/100) sia sulle linee che sui rami decisionali.

Classe Sotto Test	Copertura Linee	Copertura Rami	Metodi
AESEncryptionStrategy	100%	100%	5/5
PasswordHasher	100%	100%	3/3
KeyManager	95%	92%	7/8
UserServiceImpl	94%	88%	9/10
VaultServiceImpl	90%	85%	12/13
SafeCoreException	100%	100%	1/1

Tabella 4.3: Dettaglio della copertura per i componenti critici di business e sicurezza.

L'analisi JaCoCo conferma che **tutta la logica di business critica e i protocolli crittografici sono stati verificati con successo**. La copertura complessiva dell'88.2/100 sulle linee e dell'83.7/100 sui rami decisionali rappresenta un ottimo risultato.

La porzione di codice non coperta (inferiore al 12/100) è stata analizzata e ritenuta non critica. Essa si concentra prevalentemente in: getter e setter nelle Entity JPA, catch di eccezioni di sistema rare o gestione di animazioni secondarie o eventi asincroni JavaFX di natura estetica.

Capitolo 5

Qualità del Software

In questo capitolo viene analizzata la qualità del software SafeCore soffermandosi sulle sue caratteristiche di mantenibilità, affidabilità e performance. Nello specifico, vengono richiamati elementi già trattati nel corso dei capitoli precedenti, facendone una sintesi.

5.1 Rispetto dei Principi SOLID

L'architettura di SafeCore segue i buoni principi SOLID:

- **Single Responsibility Principle (SRP):** ogni classe ha un unico motivo per cambiare. Ad esempio, il `PasswordStrengthEvaluator` non conosce i dettagli del database; si occupa solo di calcolare un punteggio. Questo isolamento ha reso il testing unitario estremamente semplice e mirato.
- **Open/Closed Principle (OCP):** il sistema è facile da estendere ma difficile da rompere. Ad esempio grazie all'interfaccia `PasswordRule`, se domani volessimo aggiungere un controllo nuovo (magari per verificare se una password è stata hackerata in passato), basterebbe creare una nuova classe.

Gran parte del design progettuale che implementa questi principi è stato ampiamente descritto nel capitolo 3, al quale si rimanda per la lettura.

5.2 Analisi delle Performance

Sebbene SafeCore sia un'applicazione desktop, la velocità delle operazioni crittografiche è critica per non degradare l'esperienza utente. Un ritardo eccessivo nello sblocco del vault verrebbe percepito come un malfunzionamento.

Operazione	Tempo Medio (ms)	Esito Qualitativo
Cifratura AES-256-CBC	45	Eccellente
Decifratura AES-256-CBC	48	Eccellente
Derivazione Chiave (PBKDF2)	120	Ottimale*
Validazione Password	8	Istantaneo

Tabella 5.1: Performance medie rilevate in fase di test.

**Nota: Il tempo di derivazione della chiave è volutamente più alto per ostacolare attacchi di forza bruta (Brute Force Protection).*

5.3 Requisiti Funzionali e Non Funzionali

Tutti i requisiti, distinti in funzionali e non funzionali, sono stati esplicitati all'interno del Capitolo 2, a cui si rimanda per la lettura.

In particolar modo, le specifiche dell'analisi qualitativa relativa allo **Standard ISO/IEC 25010** è stata descritta in maniera esaustiva al Paragrafo 2.2.3, al quale si rimanda per la lettura.

5.4 Metriche di mantenibilità

Grazie all'uso di strumenti di analisi statica (JaCoCo), sono state estratte le seguenti metriche:

- **Copertura del Codice:** è stata raggiunta una copertura superiore all'85%. L'argomento è stato già trattato alla sezione 4.3, alla quale si rimanda per la lettura.
- **Complessità Ciclomatica:** mantenuta mediamente sotto il valore di 10 per metodo. Questo garantisce che il codice sia leggibile anche da sviluppatori non coinvolti nel progetto iniziale.